

Adaptive QoS Management in OneM2M Standard: Machine Learning and Deep Learning for IoT Network Optimization

Jamal ET-TOUSY

`j.ettousy.ced@uca.ac.ma`

Cadi Ayyad University

Abdellah ZYANE

Cadi Ayyad University

Saeed SHARIF

University of East London

Research Article

Keywords: OneM2M, IoT, QoS optimization, Machine Learning, Deep Learning, MAPE-K framework, Traffic offloading, Overload management, API orchestration

Posted Date: November 5th, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-7987618/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.
[Read Full License](#)

Additional Declarations: No competing interests reported.

Version of Record: A version of this preprint was published at Journal of Network and Systems Management on May 6th, 2026. See the published version at <https://doi.org/10.1007/s10922-026-10071-4>.

Adaptive QoS Management in OneM2M Standard: Machine Learning and Deep Learning for IoT Network Optimization

ET-TOUSY Jamal¹ | ZYANE Abdellah¹ | SHARIF Saeed²

¹ LAPSSII Laboratory, High School of Technology of Safi (EST Safi), Cadi Ayyad University, Morocco
B.P. 89, Route Dar Si Aïssa, 46000, Safi, Morocco

² Intelligent Technologies Research Group, Department of Computer Science and Digital Technology,
ACE, UEL, London E16 2RD, UK.

Abstract

The exponential growth of Internet of Things (IoT) deployments has created unprecedented challenges in maintaining optimal Quality of Service (QoS) while managing dynamic resource allocation in IoT middleware platforms. Although OneM2M, standardized and adopted by ETSI (European Telecommunications Standards Institute), provides a robust framework for IoT interoperability, it does not inherently address overload scenarios and dynamic resource optimization challenges. This paper presents a comprehensive machine learning and deep learning framework for intelligent traffic offloading and QoS optimization within the OneM2M standard architecture. The primary objective is to optimize QoS metrics through a dual approach: first, traffic-oriented metrics including Round-Trip Time (RTT) and Success Rate, followed by resource-oriented metrics encompassing CPU and RAM utilization, enabling proactive system adaptation to prevent performance degradation. We developed an autonomous system based on the MAPE-K (Monitor, Analyze, Plan, Execute - Knowledge) framework that continuously monitors and collects critical QoS data from Azure IoT devices streaming real-time sensor data through Event Hubs to OneM2M Common Services Entity (CSE) servers, establishing performance thresholds for normal, acceptable, and critical operational states. The collected QoS data undergoes comprehensive preprocessing to address class imbalance using SMOTE oversampling techniques and feature engineering of composite metrics. Eight machine-learning algorithms were systematically evaluated, including Random Forest, LightGBM, XGBoost, and Support Vector Machines, followed by three deep learning approaches: Convolutional Neural Networks (CNN) for spatial pattern recognition, Long Short-Term Memory (LSTM) networks for temporal dependencies, and Variational Autoencoders (VAE) for feature compression. The optimized models are deployed through a scalable API infrastructure hosted on Hugging Face that orchestrates the entire decision-making process, serving as the central intelligence hub for receiving real-time QoS data, processing it through trained models, and providing autonomous decision recommendations for traffic management and resource allocation. The optimized Random Forest model achieved 96% accuracy with perfect precision for critical state detection, while ensemble approaches reached 98% accuracy, demonstrating RTT reduction of 30-50%, CPU/RAM usage optimization of 20-30%, and maintaining success rates above 90% across diverse traffic patterns (uniform, real-time, and burst scenarios), which proves particularly beneficial for critical IoT domains such as e-health applications where system reliability and responsiveness are paramount. This research fundamentally addresses critical limitations in current OneM2M implementations by introducing autonomous overload management capabilities, establishing a robust foundation for next-generation IoT infrastructure with promising extensions toward federated learning approaches for distributed model training across heterogeneous IoT networks, seamless integration with 5G/6G network slicing for enhanced QoS guarantees, and deployment in industrial IoT environments requiring ultra-low latency and high reliability standards.

Keywords: OneM2M, IoT, QoS optimization, Machine Learning, Deep Learning, MAPE-K framework, Traffic offloading, Overload management, API orchestration.

1 | INTRODUCTION

The Internet of Things (IoT) ecosystem has witnessed unprecedented growth, fundamentally transforming how devices interact and communicate in our interconnected world. Recent market analysis indicates that the number of connected IoT devices is projected to reach 18.8 billion globally in 2024 [1], representing a 13% growth rate, while the global IoT device management market is expected to reach USD 45 billion by 2033, growing at a CAGR of 32% [2]. This explosive growth is driven by the increasing adoption of Industry 4.0 technologies, smart city initiatives, and the proliferation of edge computing applications that demand real-time processing capabilities. However, this rapid expansion brings significant challenges in managing Quality of Service (QoS) across heterogeneous networks, particularly when dealing with varying traffic patterns, resource constraints, and mission-critical applications that require guaranteed performance levels [3]. Traditional static resource management approaches are proving inadequate to handle the dynamic nature of modern IoT deployments, necessitating the development of intelligent, adaptive middleware solutions that can autonomously optimize system performance while maintaining strict QoS requirements [4].

The OneM2M standard [5], developed through a collaborative effort of eight preeminent global standards organizations including ETSI [6], represents a pivotal advancement in IoT middleware standardization. OneM2M is the global standards initiative that covers requirements, architecture, API specifications, security solutions and interoperability for Machine-to-Machine and IoT technologies, providing a comprehensive framework for scalable and interoperable IoT systems. The architecture standardized by OneM2M defines an IoT Service Layer, i.e., a vendor-independent software middleware between processing and communication hardware and IoT applications providing a set of functions commonly needed by IoT applications. The OneM2M standard defines a comprehensive service layer that sits between IoT devices and applications, providing a set of common service functions (CSFs) such as device management, data storage, and security. This horizontal architecture enables seamless communication between diverse devices and systems, facilitating interoperability across different vendor ecosystems [7]. OneM2M specifications provide a framework to support applications and services such as smart grid, connected car, home automation, and public safety, demonstrating its versatility across various IoT verticals. Despite its comprehensive coverage of functional requirements, the current OneM2M architecture presents limitations in dynamic resource optimization and autonomous overload management [8], particularly when dealing with unpredictable traffic patterns and resource-intensive applications that can overwhelm the Common Services Entity (CSE) infrastructure.

Contemporary IoT middleware systems face critical challenges in maintaining consistent QoS under dynamic operational conditions, particularly when confronted with varying traffic patterns that can range from steady-state uniform loads to sudden burst scenarios. The static nature of traditional resource allocation mechanisms [9] creates significant bottlenecks when IoT systems experience unexpected load spikes, leading to degraded performance metrics including increased Round-Trip Time (RTT), reduced success rates, and suboptimal resource utilization. Mission-critical IoT applications, such as industrial automation, healthcare monitoring, and autonomous vehicle networks, require stringent QoS guarantees with specific Service Level Agreement (SLA) requirements that cannot be compromised. These applications typically demand RTT values below 2 seconds for normal operations, while tolerance levels between 2-4 seconds represent acceptable degradation, and values exceeding 4 seconds indicate critical system states requiring immediate intervention. The absence of autonomous, self-adaptive mechanisms in current IoT middleware architectures necessitates manual intervention for system optimization, which is neither scalable nor feasible for large-scale deployments. Furthermore, the heterogeneous nature of IoT traffic patterns, including uniform data streams, real-time sensor feeds, and bursty event-driven communications, requires sophisticated prediction and adaptation mechanisms that can proactively identify potential system overloads and implement preventive measures to maintain optimal performance across diverse operational scenarios.

This research aims to develop and implement a novel dual QoS optimization framework that addresses both traffic-oriented metrics (RTT and success rate) and resource-oriented parameters (RAM and CPU utilization) through the integration of Machine Learning (ML) and Deep Learning (DL) techniques within the OneM2M architecture. The primary objective centers on creating an autonomous, self-adaptive IoT middleware system that leverages the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) framework to enable intelligent decision-making based on real-time system performance analysis. The core contribution lies in the development of a production-ready ML/DL-integrated solution that combines ensemble learning techniques, including optimized Random Forest and LightGBM models, with advanced deep learning approaches such as Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM) networks, and Variational Autoencoders (VAE). The research introduces a comprehensive feature engineering methodology that creates composite metrics, and performance scores that encapsulate overall system health into interpretable indicators. Additionally,

the study presents a scalable API orchestration system deployed on Hugging Face infrastructure that enables real-time inference and decision-making, facilitating seamless integration with existing OneM2M deployments. The framework's effectiveness is validated through comprehensive evaluation across diverse traffic scenarios, including uniform, real-time, and burst traffic patterns, demonstrating significant improvements in system performance with RTT reductions of up to 50% and consistent maintenance of success rates above 90% even under high-load conditions.

The remainder of this paper is organized as follows. Section 2 reviews the OneM2M architecture, recent literature, QoS management techniques, and AI integration gaps. Section 3 details the system design, data collection, and preprocessing steps. Section 4 presents ML/DL models, ensemble strategies, and deployment via Hugging Face API. Section 5 reports results under uniform, burst, and real-time traffic. Section 6 concludes with key contributions and future directions.

2 | RELATED WORKS

2.1 | OneM2M Standard and Architecture

Established in 2012, the oneM2M standard is a global initiative led by major Standards Development Organizations (SDOs) such as ETSI (Europe), ATIS (USA), TTA (Korea), and others. Its primary mission is to create a unified service layer for Machine-to-Machine (M2M) and Internet of Things (IoT) applications, promoting interoperability across different industries, technologies, and countries. The standard addresses the fragmentation in the IoT ecosystem by offering a common set of specifications that enable seamless communication between devices, platforms, and services [10].

The oneM2M architecture (See **Figure 1**) is structured into three functional layers: the Application Layer, the Service Layer, and the Network Layer. The **Application Layer** hosts the Application Entities (AEs), which represent user-facing IoT services such as smart homes, connected vehicles, or industrial automation. These AEs interact with the underlying service layer through the Mca interface to access necessary functionalities [11].

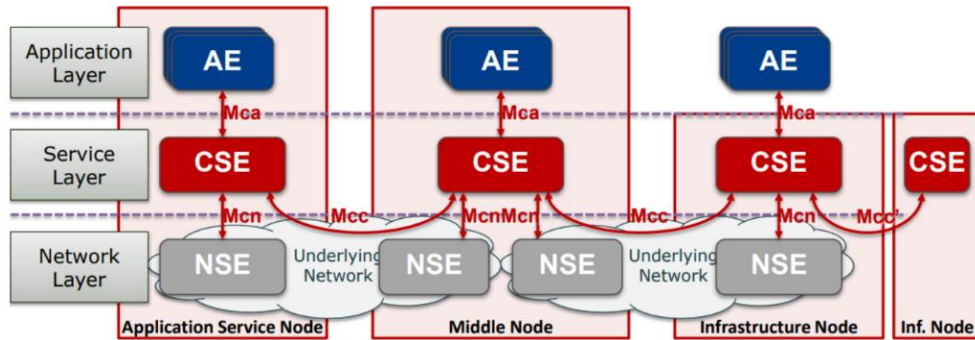


Figure 1: Architecture of oneM2M platform [12]

The **Service Layer** is the core of the oneM2M platform. It houses the Common Services Entity (CSE), which provides a rich set of functions for data exchange, device and service management, security, and more. This layer is responsible for ensuring that application-level interactions are consistent and standardized, regardless of the type of device or network involved.

The **Network Layer** includes the Network Services Entity (NSE), which manages communication with the underlying transport network. It ensures that messages are delivered between nodes through the Mcn reference point, enabling the service layer to remain network-agnostic.

As shown in **Figure 1**, oneM2M defines different node types within its architecture. The **Application Service Node (ASN)** contains both an AE and a CSE and is typically located at the edge of the network, close to the end-user. The **Middle Node (MN)** acts as a relay or aggregator between the edge and the core of the network, forwarding data and requests. The **Infrastructure Node (IN)** is responsible for core operations such as system-wide data storage, management, and control. Communication between CSEs in different nodes is handled by the Mcc interface, allowing distributed coordination and data consistency.

The CSE is a critical component in oneM2M and supports a wide array of **Common Service Functions (CSFs)**. These CSFs (See **Figure 2**) are standardized capabilities provided by the platform to simplify and support core IoT operations across heterogeneous environments. In brief:

- ✓ **Registration** allows devices and applications to join the system securely.
- ✓ **Security** ensures proper authentication, authorization, and data protection.
- ✓ **Communication Management** handles message delivery and session coordination.
- ✓ **Data Management & Repository** organizes and stores structured data for easy access.
- ✓ **Subscription & Notification** provides mechanisms for event-based communication.
- ✓ **Discovery** enables the dynamic location of services and resources.
- ✓ **Application & Service Management** oversees lifecycle control and deployment.
- ✓ **Device & Group Management** supports organizing, tracking, and controlling devices.
- ✓ **Semantics** facilitates interoperability through shared data meaning.
- ✓ **Additional services** such as Location, Time, Process, Session, and Network Exposure enhance context-awareness and integration.

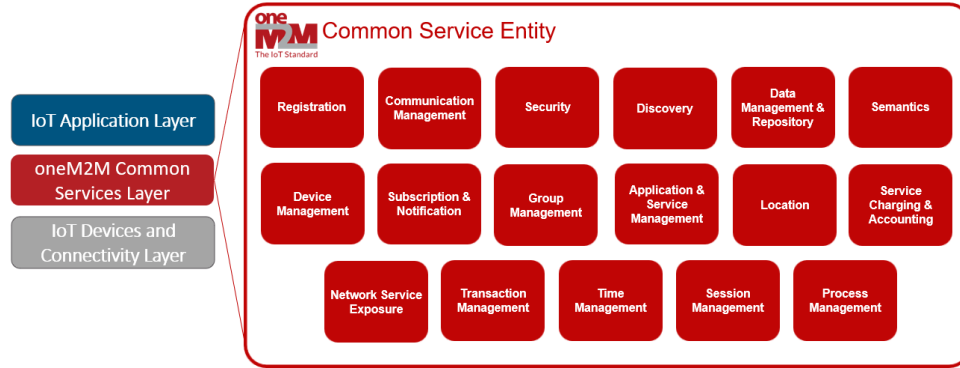


Figure 2: oneM2M Common Services Toolkit (CSE functions) [13]

The oneM2M architecture offers a standardized and modular framework that simplifies the development of interoperable and scalable IoT solutions.. The clearly defined roles of each architectural element ensure that both vertical and horizontal communication is standardized, facilitating broad interoperability in distributed and heterogeneous IoT environments.

2.2 | OneM2M Literature Review

In order to provide a comprehensive overview of the scientific literature on the oneM2M standard (See Table 1), this table brings together the majority of relevant articles published between 2020 and 2025. Our own research contributions are presented separately in the next section.

Table 1: OneM2M literature Review

Ref	Year	oneM2M Aspect	Key findings
[14]	2020	Semantic IoT Gateway based on oneM2M; RDF graph extension for resource representation; Aggregator for cross-domain interoperability	Proposes a oneM2M semantic gateway with RDF graphs for dynamic interoperability and rule management, achieving up to 40% better performance than ontology-based solutions.
[15]	2020	Protocol comparison (MQTT/CoAP); Energy efficiency; Fine-dust IoT integration	Evaluates MQTT vs. CoAP in oneM2M fine-dust sensors, finding MQTT to be most energy-efficient.
[16]	2020	Interworking Proxy Entity (IPE); Middleware for heterogeneity; Logistics scenario	Proposes a oneM2M-based IPE for connecting heterogeneous IoT devices; validated in logistics use case.
[17]	2020	Caching mechanism; Traffic load reduction; Real-world performance evaluation	Validates a caching model in a oneM2M platform, showing reduced traffic and improved response time.
[18]	2020	Security of oneM2M layer; ML-based intrusion detection; Continuous learning	Proposes an ML-based IDPS for oneM2M IoT platforms, enabling real-time multi-level threat detection.
[19]	2020	Security & IDPS for oneM2M; Edge machine learning; Three-level detection	Presents a generic ML-based IDPS for oneM2M, combining multiple detection modes, validated with high accuracy.
[20]	2021	Lightweight DTLS; Dynamic access control; Fog computing & proxy re-signcryption	Analyzes lightweight security for WSANs in OM2M, integrating DTLS, access control, and fog computing for enhanced protection.
[21]	2021	oneM2M-based IIoT; Interoperability for maintenance; ML integration	Implements predictive maintenance in smart factories, using oneM2M IIoT and Random Forest for 82%+ accuracy.
[22]	2021	Layered MN-CSE/IN-CSE; Threshold detection; Cost reduction for smart homes	Proposes a layered architecture for smart homes with oneM2M, reducing device needs and maintenance costs.

[23]	2021	IoT-MR integration; AE/CSE mapping; 3D object control	Proposes an MR-IoT platform using oneM2M for synchronized control of physical and virtual objects.
[24]	2021	Gateway architecture; Mapping to OPC UA; Industrial systems integration	Proposes a gateway for info exchange from oneM2M to OPC UA, enabling industrial IoT/automation interoperability.
[25]	2022	OM2M vs. other IoT platforms; Discovery, semantics, middleware	Benchmarks OM2M (oneM2M) vs. IoTivity/LwM2M/FIWARE, mapping strengths in discovery, semantics, and data management.
[26]	2022	oneM2M IoT-MR interoperability; MR Virtual Device; Open-source integration	Proposes architectures connecting MR and IoT via oneM2M, supporting generic device/domain interoperability and efficient MR resource management.
[27]	2022	OneM2M architecture for M2M/IoT services, data management, security, and integration with machine learning.	Reviews how oneM2M manages data flow and security in M2M systems and discusses the role of machine learning in IoT applications.
[28]	2022	RDF-based semantic annotation; oneM2M to NGSI-LD mapping; Linked data	Proposes protocol for translating oneM2M resources to semantically-annotated NGSI-LD, validated in a parking scenario.
[29]	2022	Group-based authentication; Mobius integration; Overhead reduction	Implements group-based authentication for oneM2M (Mobius), achieving up to 4x efficiency and flexible interoperability.
[30]	2022	oneM2M Mobius for SDN; Hierarchical modeling; Scalability/reactivity metrics	Evaluates oneM2M (Mobius) as an SDN controller, demonstrating scalable real-time network management.
[31]	2023	Security & threat modeling for OM2M; Access control; Real-world attack tests	Evaluates security for OM2M-based smart city IoT, with practical tests and recommendations for baseline protection.
[32]	2023	oneM2M security/authentication overview; Vulnerabilities; Categorized solutions	Reviews and categorizes authentication techniques for oneM2M, supporting robust security design.
[33]	2023	oneM2M platform standardization; 12 CSFs; REST APIs & semantic modeling	Compares oneM2M, FIWARE, Cisco Kinetic for Smart Cities, ranking oneM2M best for architecture and service functions.
[34]	2023	oneM2M DML layer; OM2M resource modeling; ACP & multi-tenant data	Proposes multi-layer Smart City architecture using oneM2M/IUDX for real-time, interoperable, secure data sharing.
[35]	2023	UAV-GCS integration; Real-time RF/LTE; Antenna tracker/web app	Real-time UAV-GCS communication via oneM2M, enabling autonomous antenna tracking and seamless connectivity.
[36]	2024	Comparative analysis of architectures; Performance metrics; Smart city scenarios	Evaluates multiple oneM2M-based architectures for IoT in Smart Cities, benchmarking performance and scalability.
[37]	2024	Interworking Proxy Entity; Modbus-to-oneM2M mapping; Data cache optimization	Architecture enables Modbus-oneM2M integration via IPE/SDT; use case shows improved performance and real-time data integration.
[38]	2024	IPE for Matter; oneM2M platform-level integration; ACME/Mobius	Presents a Matter-oneM2M IPE for seamless integration; experiments confirm negligible overhead and advanced services.
[39]	2024	Legacy device integration; oneM2M service layer; Real-world performance	Implements standardized AQM via oneM2M, integrating Wi-Fi, LoRa, Zigbee; achieves 99.5% reliability and low latency.
[40]	2024	TinyOneM2M implementation; Resource-constrained support; REST API/scalability	Presents lightweight oneM2M implementation for low-power devices, confirming low memory use and efficient M2M comms.
[41]	2025	IoT Gateway integration with oneM2M; Interoperability for urban infrastructure modeling; Hierarchical/Modular design using oneM2M DataGateway;	Proposes a DSML and DEVS-based workflow to simulate IoT infrastructures, using oneM2M for interoperability and integration of heterogeneous sensors in urban environments.

After analyzing this collection of papers, it is clear that most research focuses on **interoperability**, **security**, and **integration** aspects of the oneM2M standard. There is a strong emphasis on frameworks, semantic data models, cross-domain architectures, and security enhancements for IoT and smart city applications. However, **Quality of Service (QoS)**, including performance guarantees, latency, reliability, and resource optimization, remains underexplored both in the literature and in the oneM2M standard itself. This gap highlights the need for further research and new solutions to address QoS challenges within the oneM2M ecosystem.

2.3 | QoS Management in OneM2M

Quality of Service (QoS) plays a crucial role in IoT systems, particularly as device connectivity and traffic volumes continue to grow in domains such as eHealth, industrial automation, and smart cities [42, 43]. Despite its importance, our analysis of recent literature reveals that QoS remains an underexplored aspect within both the OneM2M standard and academic research. To address this gap, our work followed a structured, step-by-step approach.

Step 1 – Performance Evaluation [44]:

Through intensive benchmarking in *QoS in OneM2M: Performance Evaluation and Analysis*, we demonstrated that OneM2M can handle over 10,000 simultaneous device connections with message delivery rates exceeding 95% under normal load. However, under stress conditions, throughput dropped by up to 30% and latency increased beyond 500 ms, highlighting bottlenecks that can impact real-time applications.

Step 2 – Protocol Comparison [45, 46]:

Our comparative study between HTTP and MQTT in OneM2M showed that MQTT achieves up to 60% lower latency and 40% higher reliability than HTTP, particularly in bandwidth-constrained environments. Integrating CoAP alongside MQTT further improved performance for low-power IoT scenarios, confirming that protocol choice is a decisive factor for achieving robust QoS and system scalability in OneM2M platforms.

Step 3 – Loss Recovery Mechanisms [47]:

We introduced adaptive loss recovery mechanisms (such as Selective Repeat and ARQ) at the OneM2M middleware layer. Experiments revealed that Selective Repeat reduces packet loss rates by more than 50% compared to traditional Go-Back-N schemes, while MQTT with Selective Repeat consistently achieved end-to-end message reliability above 98% even in high-loss environments.

Step 4 – Autonomous QoS Management [48]:

Finally, we proposed an automated, traffic-oriented QoS management mechanism based on the MAPE-K framework. Dynamic resource adaptation, guided by real-time SLA monitoring, allowed the platform to maintain priority service for critical domains and avoid congestion. In testbeds simulating IoT workloads, this approach maintained SLA compliance above 97% and reduced critical message delays by up to 45%.

All these works are consolidated in our latest journal article [49], which examines traffic overload in OneM2M with HTTP, MQTT, and CoAP. Using queueing theory and a MAPE-K-based orchestration, the study enables SLA-driven traffic management, cloud offloading, and provides an evaluation of the financial costs to optimize QoS.

Following these steps, the objective of this paper is to optimize QoS in OneM2M through a more intelligent approach, by integrating artificial intelligence into the platform. By leveraging machine learning and deep learning techniques, our aim is to enable the OneM2M platform to predict network conditions, dynamically allocate resources, and adapt in real time to varying traffic patterns.

2.4 | AI, ML, and DL in oneM2M

Recently, the oneM2M standard has launched various initiatives, including proofs-of-concept and technical reports, to facilitate the integration of artificial intelligence (AI), machine learning (ML), and deep learning (DL) into IoT platforms. Notably, ETSI technical reports [50, 51] provide comprehensive analyses and implementation guidance for embedding AI-driven functionalities within the oneM2M architecture, demonstrating practical AI applications such as anomaly detection, fault management, and resource optimization. Moreover, the oneM2M Technical Report TR 0068 outlines the architectural enhancements required for supporting AI and ML services within the oneM2M framework, paving the way for standardized, interoperable intelligent IoT solutions [52]. As illustrated in **Figure 3**, the oneM2M system should be enhanced with a new common service function (CSF) to support AI capabilities, a set of new resources, and various AI/ML use cases.

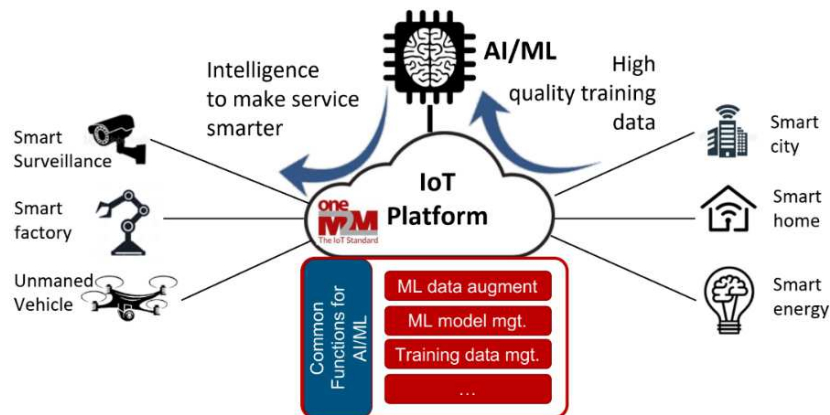


Figure 3: Integration of AI/ML in oneM2M IoT platforms to enable smarter services [53].

Academic literature also highlights several applications leveraging AI and ML in the context of oneM2M platforms. Research has shown the utility of deep learning models, such as bidirectional LSTM, in accurately predicting environmental parameters within oneM2M-enabled industrial IoT setups [54]. Additionally, machine learning-based intrusion detection systems specifically tailored

for oneM2M service layers have been effectively proposed and validated, demonstrating the feasibility and necessity of intelligent security mechanisms within IoT ecosystems [55]. The combination of fog computing with oneM2M to enable AI services closer to the edge also highlights the platform's adaptability to distributed AI implementations [56]. Despite these advances, there remains a notable scarcity of research explicitly focused on the optimization of Quality of Service (QoS) within oneM2M through AI-driven methodologies. Although preliminary investigations using ML for DDoS attack detection have provided insights into QoS improvement under stress conditions [57], comprehensive studies addressing systematic QoS optimization via AI, particularly concerning real-time resource allocation, predictive analytics, and dynamic network management, are still limited.

3 | SYSTEM ARCHITECTURE AND DESIGN

The objective of this section is to elaborate on our procedure for integrating machine learning and deep learning into the oneM2M platform. We detail the complete workflow from data injection, collection, and preprocessing, to facilitate the effective training of predictive models. These models enable efficient offloading strategies, thereby maintaining optimal conditions for both traffic-oriented and resource-oriented parameters.

3.1 | Overall System Architecture

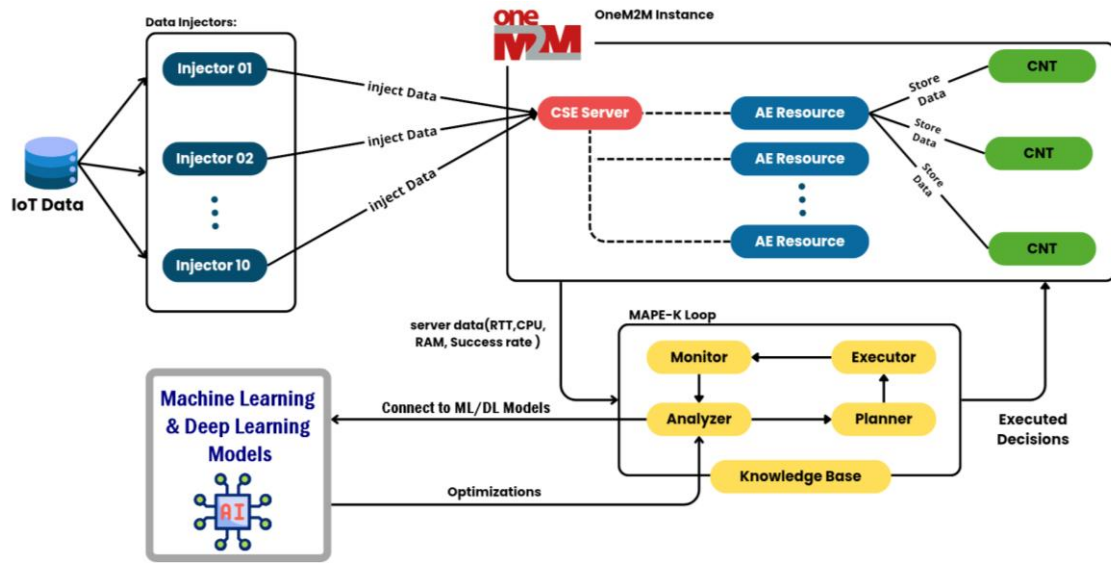


Figure 4: System architecture combining OneM2M, ML/DL models, and MAPE-K for QoS optimization.

Our proposed architecture (See Figure 4) is composed of multiple interconnected components designed to integrate machine learning (ML) and deep learning (DL) techniques within the oneM2M framework.

➔ Data Collection:

Initially, data collection involves IoT sensors interfaced through Azure IoT, capturing diverse real-world measurements such as temperature, humidity, ambient light, barometric pressure, sound noise, total volatile organic compounds (eTVOC), discomfort index, and heat stroke index (See Figure 5). These data streams are aggregated in real-time using Azure IoT Central, which serves as a primary data collection point, crucial for accurate Quality of Service (QoS) assessment.

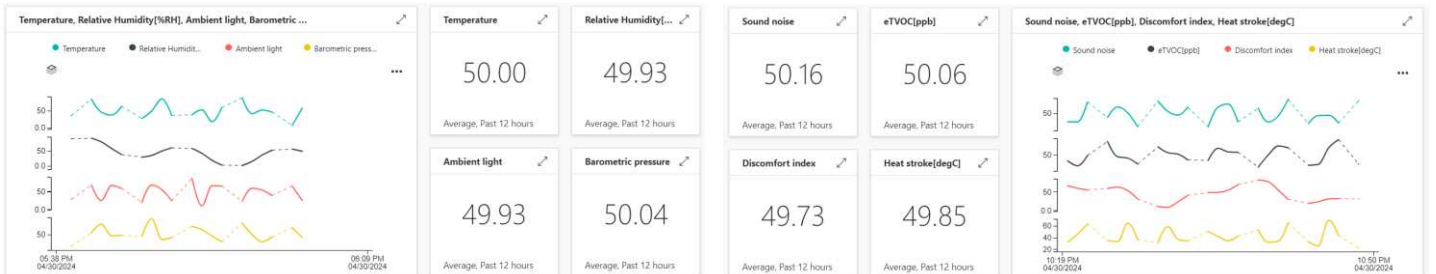


Figure 5: Example of real-time environmental data collected via Azure IoT sensors

➔ Data Transmission:

Subsequently, the aggregated data is transmitted via Azure Event Hubs, acting as a messaging broker to efficiently route the data toward the downstream system components. The data injectors, implemented using Azure's Python SDK, consume and preprocess

this data before forwarding it to the oneM2M Common Service Entity (CSE) server. Once received by the CSE server, data is stored within Application Entity (AE) resources in dedicated containers to facilitate further analysis.

➡ QoS Monitoring and Load Testing:

Throughout this data pipeline, key QoS metrics are continuously monitored to evaluate system performance under varying load conditions. To test and validate the system's performance and scalability, we designed an experimental scenario that progressively increases the data injection volume. Initially, ten injectors transmit data at a rate of 100 requests per second, with data volumes gradually increased and intervals between requests correspondingly reduced. This stress-testing approach ensures a robust evaluation of the oneM2M platform's ability to handle increasing traffic loads.

➡ MAPE-K Methodology and ML/DL Integration:

We employed the MAPE-K (Monitor, Analyze, Plan, Execute, and Knowledge) loop methodology to systematically gather QoS data. The MAPE-K framework provides structured and adaptive monitoring capabilities that enable real-time data collection and analysis. The collected data is then leveraged to select and train appropriate ML and DL models. These models are essential for making informed offloading decisions, thus optimizing system performance and ensuring resource-oriented and traffic-oriented parameters are maintained within desirable thresholds. Further details on the data collection and preprocessing techniques employed will be presented in the subsequent dedicated section.

3.2 | Data Collection and Preprocessing

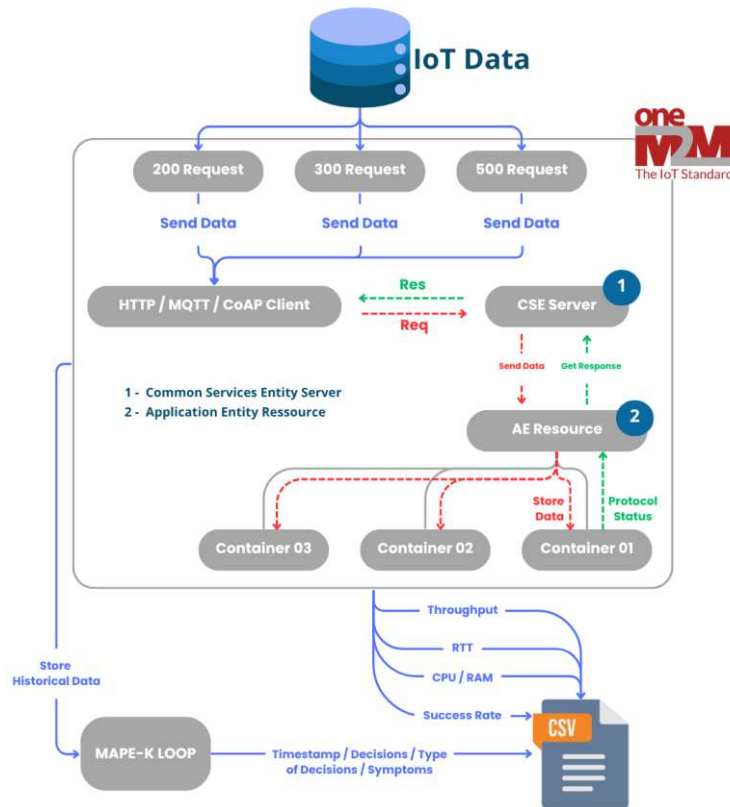


Figure 6: Data flow and monitoring in OneM2M with multiple protocols and MAPE-K integration.

3.2.1 | Data Collection Using MAPE-K Loop :

Our dataset (See Figure 7) collection follows a structured, adaptive, and rigorous approach utilizing the MAPE-K loop (Monitor, Analyze, Plan, Execute, Knowledge). This architectural framework (See Figure 6) facilitates real-time monitoring and predictive QoS management within the oneM2M platform.



	timestamp	rtt	cpu	ram	success rate	latency	throughput	symptom_encoded	decision_encoded	decision_type_encoded
0	1.723920e+09	1.132978	72.800000	94.300000	100.000000	566.488981	0.702244	2	1	1
1	1.723921e+09	2.439391	37.855471	89.738202	99.755005	867.799190	0.871052	1	1	2
2	1.723920e+09	0.586336	62.700000	92.200000	100.000000	293.168187	0.739408	2	1	1
3	1.723920e+09	1.327147	70.500000	92.600000	100.000000	663.573742	0.717494	2	1	1
4	1.723921e+09	2.623520	67.750285	91.052162	85.645689	1521.983019	0.771558	0	0	0

Figure 7: Example of the collected dataset

➡ Monitor:

The Monitor component continuously collects comprehensive real-time QoS metrics from the oneM2M platform, including:

- **Round-Trip Time (RTT):** Evaluating the latency between requests and responses.
- **CPU usage:** Assessing the computational load on the server.
- **RAM utilization:** Monitoring memory consumption as data volume increases.
- **Success Rate:** Determining the percentage of successfully processed requests.
- **Latency:** Measuring processing delays under system load.
- **Throughput:** Calculating the rate of successfully processed requests, reflecting system capacity.
- **Timestamp:** Providing temporal context to performance metrics.

These metrics are systematically captured during simulated IoT scenarios with varying request loads (e.g., 200, 300, 500 requests), enabling detailed analysis of system performance under different stress conditions.

➡ Analyze:

The Analyzer component classifies collected data into clearly defined QoS categories:

- **Preferable (0):** Optimal system performance, characterized by $RTT < 2s$, low CPU and RAM usage, and high success rates.
- **Acceptable (1):** Moderate performance degradation with RTT between 2s and 4s, resource usage within operational limits.
- **Critical (2):** Severe system congestion and overload conditions, signified by RTT exceeding 4s, success rates below 90%, or CPU/RAM usage exceeding 80%.

➡ Plan and Execute:

Upon detection of potential performance issues, the Planner component formulates adaptive strategies, including resource reallocation, traffic offloading, or rerouting. Subsequently, the Executor component implements these strategies dynamically, ensuring seamless adaptation and minimizing performance disruptions.

➡ Knowledge:

All monitored metrics, analytical insights, decisions, and their outcomes are recorded in historical CSV datasets. This comprehensive knowledge base facilitates ongoing learning and model refinement, enhancing future predictive capabilities and decision-making efficiency.

3.2.2 | Data Preprocessing

To ensure robust model training, a series of meticulous preprocessing steps were conducted:

➡ Missing Value Management:

Although the data collection was carried out in a controlled OneM2M environment, a thorough inspection was conducted to verify data completeness. Records containing more than 20% missing values were excluded to prevent model degradation due to unreliable inputs. For entries with minimal missing data, median imputation was applied. This approach preserved the statistical properties of the dataset while minimizing the influence of outliers, thus maintaining overall data integrity.

➡ Data Cleaning:

General data cleaning operations were implemented to ensure consistency and eliminate redundancy. These included standardizing data formats, removing duplicate entries, and validating the temporal order of records using timestamps. Such procedures enhanced the structural quality of the dataset and reduced noise, contributing to better model generalization.

➡ Feature Scaling:

To ensure uniformity across input features and improve algorithmic efficiency, all numerical variables were normalized using the MinMaxScaler technique. This transformation scaled the values into the range $[0,1]$, which helped prevent features with larger magnitudes from dominating the learning process and facilitated faster convergence during training.

3.2.3 | Class Imbalance and Data Balancing

During the data processing stage, a substantial imbalance was identified among the decision classes (**See Figure 8**), which raised concerns about the model's ability to generalize and detect rare yet critical system states. The dataset includes three primary decision categories:

- **Keep Local** (encoded as 0): representing 48.14% of the samples
- **Partial Switch to Cloud** (encoded as 1): accounting for 42.22%
- **Full Switch to Cloud** (encoded as 2): comprising only 9.64%

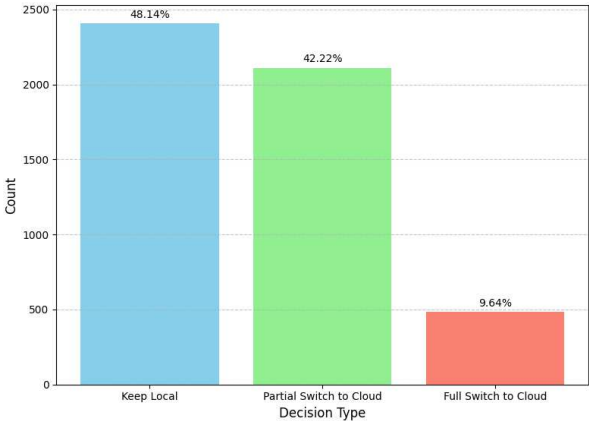


Figure 8: Class distribution of decision types

This uneven distribution, particularly the underrepresentation of the “Full Switch to Cloud” class, posed a significant risk of model bias. In such cases, classifiers tend to favor majority classes, resulting in poor sensitivity to minority class predictions. To address this issue and enhance classifier performance, several data balancing techniques (**See Table 2**) were applied and evaluated. The techniques included Synthetic Minority Over-sampling Technique (SMOTE), Adaptive Synthetic Sampling (ADASYN), Random OverSampling, Random UnderSampling, and a hybrid method combining SMOTE with Tomek Links (SMOTETomek). Each method introduces a distinct strategy for modifying class distributions during training:

Table 2: Overview of data balancing techniques used

Technique	Type	Description
SMOTE	Over-sampling	Generates synthetic samples for the minority class based on k-nearest neighbors.
ADASYN	Over-sampling	Focuses on creating synthetic data for harder-to-learn minority instances.
RandomOverSampler	Over-sampling	Randomly replicates minority samples to achieve balance.
RandomUnderSampler	Under-sampling	Randomly removes samples from majority classes.
SMOTETomek	Hybrid	Combines SMOTE with Tomek Links to both generate and clean minority samples.

To assess the effectiveness of each method, classification models were trained on datasets rebalanced using these techniques. Their performance was evaluated using multiple metrics, including Accuracy, Area Under the Curve (AUC), Macro Precision, Macro Recall, and Macro F1-Score. The results are summarized in the following **Table 3**:

Table 3: Performance comparison of balancing techniques on classification metrics

Method	Accuracy	AUC	Macro Precision	Macro Recall	Macro F1
Original (Unbalanced)	0.9587	0.9895	0.97	0.97	0.97
SMOTE	0.9580	0.9900	0.97	0.97	0.97
ADASYN	0.9580	0.9907	0.97	0.97	0.97
RandomOverSampler	0.9587	0.9897	0.97	0.97	0.97
RandomUnderSampler	0.9447	0.9865	0.96	0.96	0.96
SMOTETomek	0.9580	0.9886	0.97	0.97	0.97

The comparative results indicate that all over-sampling and hybrid techniques yielded consistent improvements, especially in terms of maintaining balanced precision and recall across classes. Among these, **SMOTE** demonstrated the best trade-off between

performance stability and computational efficiency, achieving the highest AUC (0.9900) alongside high macro-averaged metrics. Although ADASYN slightly outperformed SMOTE in terms of AUC, SMOTE was favored due to its lower variance during repeated runs and broader adoption in similar contexts.

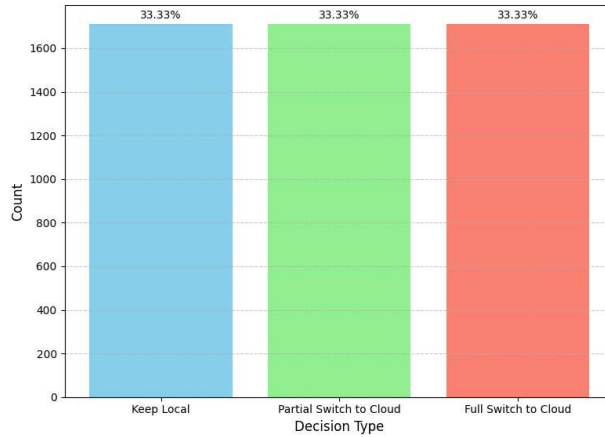


Figure 9: Class distribution of decision types after data balancing

Based on these findings, SMOTE was selected as the preferred technique to rebalance the dataset. After rebalancing (See Figure 9), each class was adjusted to include exactly **1711 samples**, resulting in a perfectly balanced distribution of **33.33% per class**. This correction effectively eliminated the class imbalance issue, ensuring that the models trained on this dataset could learn equitably from all decision types and maintain strong predictive capabilities across both frequent and rare system conditions.

4 | MODEL DEPLOYMENT AND API ORCHESTRATION

In this section, we present the progressive stages of integrating intelligent models into the OneM2M platform. First, we detail the integration of various machine learning algorithms, each evaluated within the context of our specific QoS-oriented scenario. Following this, we introduce an ensemble learning strategy aimed at combining the strengths of individual classifiers to achieve more robust and accurate decision-making. We then describe the deep learning models that were explored to capture complex, nonlinear dependencies and time-based patterns within the data. Finally, the best-performing models are encapsulated within a lightweight API, enabling real-time orchestration and intelligent decision-making in the OneM2M architecture. This deployment empowers the system to dynamically manage resources and prevent congestion or unnecessary offloading by proactively responding to traffic conditions.

4.1 | Machine Learning Integration

To optimize QoS management within OneM2M, we initially explored a set of well-established machine learning classifiers known for their effectiveness in multi-class classification problems and compatibility with real-time inference constraints. The selected models were:

- **Random Forest (RF):** A highly interpretable and robust ensemble method based on decision trees. It is well-suited for handling tabular QoS data with both numerical and engineered features. Its built-in feature importance metric was also instrumental in assessing the relevance of QoS indicators.
- **Support Vector Classifier (SVC):** Chosen for its ability to model non-linear decision boundaries through kernel tricks. It is effective in high-dimensional spaces, although computationally heavier in large datasets.
- **K-Nearest Neighbors (KNN):** A simple yet powerful distance-based method used here to serve as a baseline. It is sensitive to feature scaling and class imbalance, thus benefiting from our preprocessing efforts.
- **XGBoost:** A gradient boosting framework designed for speed and performance. It provides regularization to prevent overfitting and has shown state-of-the-art performance in numerous structured data competitions.
- **LightGBM:** A highly efficient gradient boosting algorithm optimized for speed and low memory usage. Its histogram-based learning and support for categorical features make it ideal for large-scale training in production settings.

After selecting five supervised learning algorithms, we conducted an extensive evaluation to determine their suitability for intelligent QoS orchestration in OneM2M. The following subsections analyze their outputs in terms of prediction reliability and class-wise discrimination.

To assess the classification behavior of each model, we generated confusion matrices. These matrices provide a visual summary of prediction accuracy per class by showing the number of true labels versus predicted labels. Diagonal elements represent correct classifications, while off-diagonal elements indicate misclassifications.

- **Random Forest, XGBoost, and LightGBM** exhibit strong diagonal dominance, reflecting a high number of accurate predictions across all three decision classes.
- In contrast, **SVC** shows misclassification between class 2 and other classes, especially with 130 instances of class 2 misclassified as class 1.
- **KNN** also reveals confusion between classes 0 and 1, with 103 misclassified samples from class 0.

These matrices (See Figure 10) highlight not only the overall performance but also model tendencies to confuse specific decision types—particularly critical in systems where "Full Switch to Cloud" (class 2) errors can lead to service degradation.

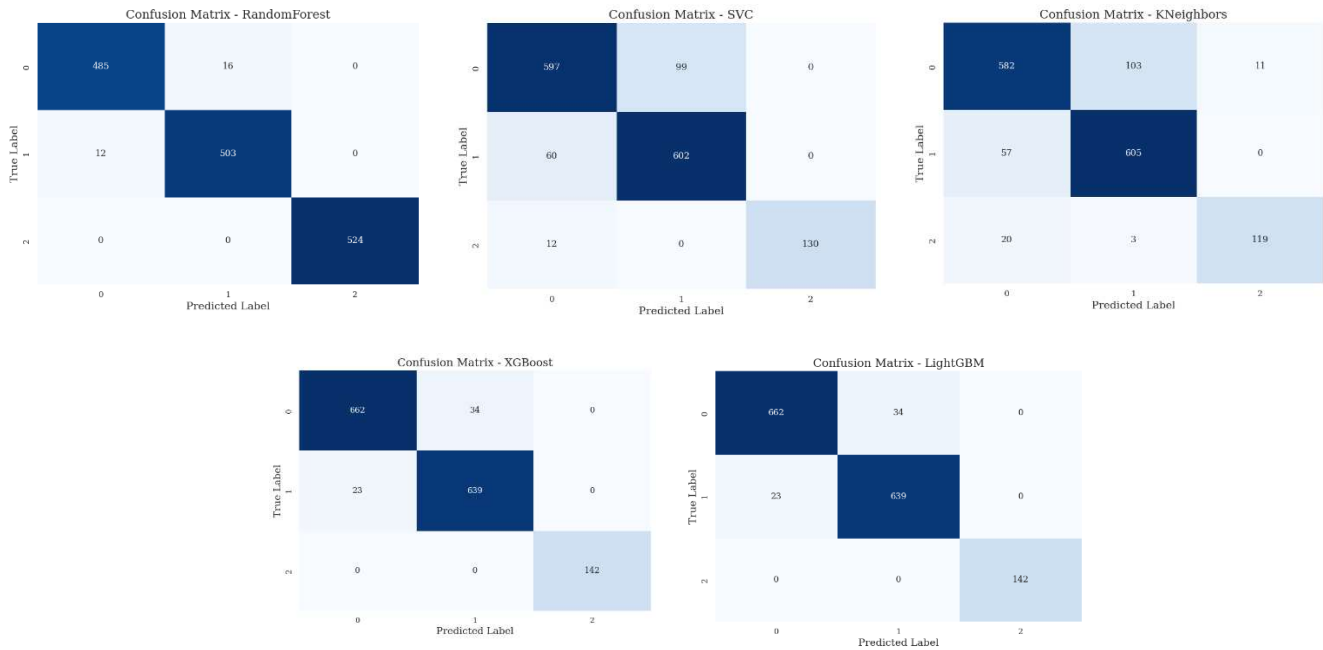


Figure 10: Confusion matrices for RF, SVC, KNN, XGBoost, and LightGBM models

Receiver Operating Characteristic (ROC) curves (See Figure 11) were plotted for each model to evaluate the true positive rate (TPR) against the false positive rate (FPR) at various classification thresholds. The Area Under the Curve (AUC) was calculated for each class:

- **Random Forest, XGBoost, and LightGBM** achieved nearly perfect AUC values (≈ 1.00), indicating excellent class separability.
- **SVC and KNN** performed acceptably, but with reduced AUC scores for classes 0 and 1, suggesting lower reliability in distinguishing these classes under variable conditions.

ROC curves are particularly useful in imbalanced or multi-class settings, where a single accuracy value may not fully capture a model's discriminatory ability. A higher AUC signifies that the model has a strong ability to distinguish between the classes across different thresholds.

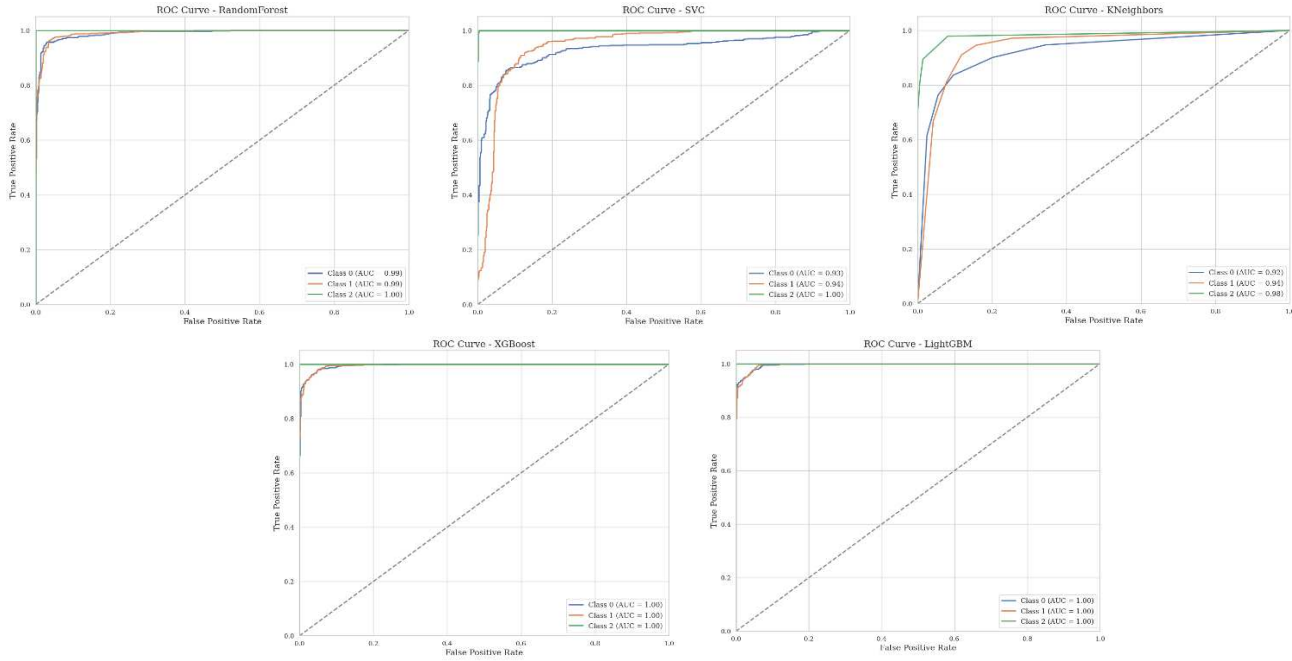


Figure 11: ROC curves for RF, SVC, KNN, XGBoost, and LightGBM models

The table 4 presents the detailed quantitative results of each classifier using common evaluation metrics:

Table 4: performance metrics of classification models across decision classes and global averages.

Model	Class 0 Precision	Class 1 Precision	Class 2 Precision	Accuracy	Macro Avg Precision	Macro Avg Recall	Macro Avg F1-Score
RandomForest	0.98	0.97	1.00	0.98	0.98	0.98	0.98
SVC	0.89	0.86	1.00	0.89	0.92	0.89	0.90
KNeighbors	0.88	0.85	0.92	0.87	0.88	0.86	0.87
XGBoost	0.97	0.95	1.00	0.96	0.97	0.97	0.97
LightGBM	0.97	0.95	1.00	0.96	0.97	0.97	0.97

These metrics are defined as follows:

- ✓ **Accuracy** reflects the overall proportion of correctly predicted instances across all classes. It is useful for general performance evaluation but may be misleading in cases of class imbalance.
- ✓ **Precision** measures the ratio of true positives to total predicted positives. It quantifies how reliable the positive predictions are, with high precision indicating few false positives.
- ✓ **Recall** (also known as sensitivity) indicates the proportion of actual positive instances that were correctly identified. A high recall implies low false negatives.
- ✓ **F1-Score** is the harmonic mean of precision and recall, providing a balanced measure especially useful when classes are imbalanced or when both false positives and false negatives are costly.
- ✓ **Macro-Averaged Metrics** compute the average of the metric independently for each class, without taking class imbalance into account. This ensures equal importance is given to all classes during evaluation, which is crucial in QoS scenarios involving critical yet less frequent states like full system overload.

In summary, Random Forest emerged as the best-performing model, closely followed by XGBoost and LightGBM. These three classifiers demonstrated both high individual class precision and macro-level stability, making them strong candidates for integration into the intelligent orchestration layer of the OneM2M system.

4.2 | Ensemble learning Approach

Based on the comparative evaluation presented in the previous section, the **Random Forest** and **LightGBM** models emerged as the top performers, demonstrating superior classification accuracy and balanced behavior across all decision classes. To further enhance their predictive capabilities, both models were fine-tuned using **Grid Search**, a hyperparameter optimization method that exhaustively explores the best parameter combinations.

Optimized Hyperparameters for LightGBM (See Figure 12): The LightGBM model was configured using the 'dart' boosting method, which allows for dropouts in boosting iterations to reduce overfitting. Key parameters such as learning rate, maximum tree depth, number of estimators, and the number of leaves were carefully adjusted, as shown below:

```
light_gbm_params = {
    'boosting_type': 'dart',
    'learning_rate': 0.1,
    'max_depth': 20,
    'n_estimators': 300,
    'num_leaves': 50
}
```

Figure 12: Grid Search Parameters for LightGBM Optimization

Optimized Hyperparameters for Random Forest (See Figure 13): Similarly, the Random Forest classifier was optimized to enhance its generalization ability and reduce variance. Parameters such as the number of trees, tree depth, minimum samples required for splitting and leaf nodes, and random seed were tuned using Grid Search:

```
rf_params = {
    "n_estimators": 300,
    "max_depth": 10,
    "min_samples_split": 10,
    "min_samples_leaf": 1,
    "random_state": 42
}
```

Figure 13: Grid Search Parameters for Random Forest Optimization

Ensemble learning is a powerful paradigm that combines multiple models to produce more robust, accurate, and generalizable predictions. In our work, we explored two ensemble strategies: **Voting** and **Stacking**, both built on top of the previously optimized **Random Forest** and **LightGBM** classifiers.

- In the **Voting Classifier**, predictions from the base learners are aggregated, and the final decision is made by majority voting (hard voting). This approach reduces variance and compensates for weaknesses in individual models.
- In the **Stacking Classifier**, a **meta-model** is trained to learn from the predictions of the base models, capturing higher-level patterns and correlations that may not be evident in individual classifiers.

Both strategies aim to enhance model robustness and achieve superior performance across all decision classes, especially in critical IoT scenarios such as traffic overload prediction and QoS adaptation.

4.2.1 | Voting Approach

The Voting Classifier combines the outputs of Random Forest and LightGBM by taking the most frequently predicted class as the final decision. This simple yet effective strategy significantly stabilizes the predictions and helps mitigate the risks of overfitting and misclassification, particularly in edge cases.

The model achieved an overall accuracy of **98%**, with strong performance across all classes (See Table 5):

Table 5: Classification Report for Voting Classifier

Class	Precision	Recall	F1-Score	Support
0	0.97	0.97	0.97	501
1	0.97	0.97	0.97	515
2	1.00	1.00	1.00	524
Accuracy			0.98	1540
Macro Avg	0.98	0.98	0.98	1540
Weighted Avg	0.98	0.98	0.98	1540

The confusion matrix (See Figure 14) confirms the model's reliability, showing minimal misclassification across all three classes:

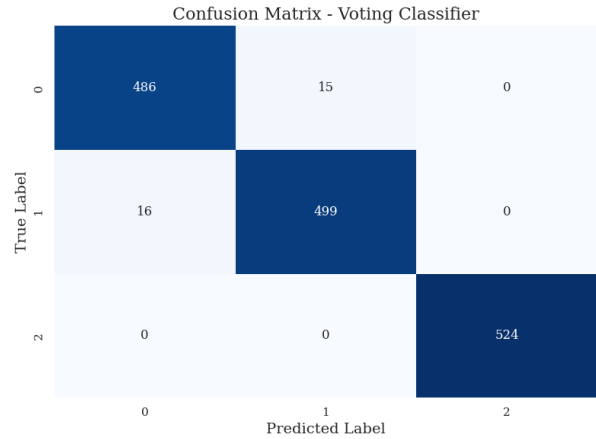


Figure 14: Confusion Matrix: Voting Classifier

4.2.2 | Stacking Approach

The Stacking Classifier employs the same base models (Random Forest and LightGBM), but instead of relying on a simple vote, their predictions are passed as input to a secondary **meta-classifier** (often logistic regression or another tree-based model). This model learns how to optimally combine the outputs of the base classifiers to refine final predictions.

The stacking strategy also reached **98% accuracy**, with performance nearly identical to the Voting Classifier (See Table 6):

Table 6: Classification Report for Stacking Classifier

Class	Precision	Recall	F1-Score	Support
0	0.97	0.97	0.97	501
1	0.97	0.97	0.97	515
2	1.00	1.00	1.00	524
Accuracy			0.98	1540
Macro Avg	0.98	0.98	0.98	1540
Weighted Avg	0.98	0.98	0.98	1540

As shown in the confusion matrix (See Figure 15), the stacking approach slightly reduced the number of misclassifications compared to the voting model, confirming the benefit of learning from the combination patterns:

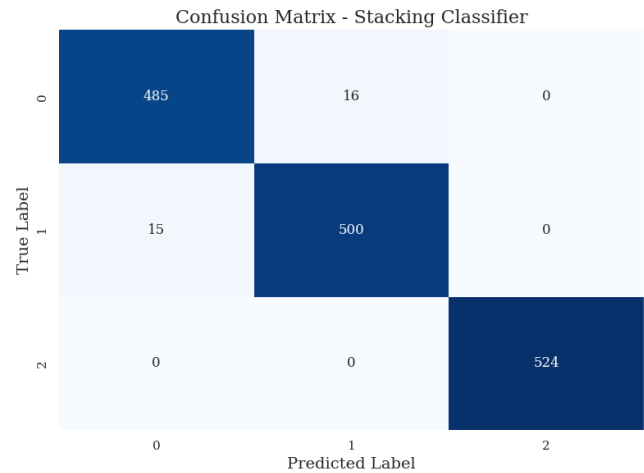


Figure 15: Confusion Matrix: Stacking Classifier

4.3 | Deep Learning Integration

To further enhance the system's ability to model complex and non-linear patterns in the QoS data, a deep learning (DL) approach was adopted. Three neural network architectures were explored: **Convolutional Neural Network (CNN)**, **Long Short-Term Memory (LSTM)**, and a **Variational Autoencoder (VAE)** model. The CNN model was employed to capture spatial relationships between features by converting tabular QoS metrics into image-like representations. Its architecture included convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected layers for classification. The LSTM model was used to capture temporal dependencies in the sequential QoS data, such as variations in RTT or CPU usage over time, making it particularly suitable for time-series modeling. Finally, the Autoencoder model was utilized to learn a compact and denoised latent representation of the QoS features, which was then passed to a Random Forest classifier for final decision-making. These deep learning models were introduced not only to improve predictive accuracy, but also to develop architectures capable of generalizing to previously unseen traffic scenarios—thereby enhancing the intelligence and autonomy of the OneM2M platform.

4.3.1 | CNN Model:

To harness the spatial characteristics inherent in Quality of Service (QoS) metrics, a novel deep learning approach was proposed using a Convolutional Neural Network (CNN). This model transforms structured data samples into grayscale images (see **Figure 16**), enabling the CNN to exploit local spatial hierarchies and feature correlations that traditional models often miss. Figure 1 illustrates several Transformed QoS samples for all decision types (0, 1, and 2), each represented as a 2D grayscale image encoding normalized feature values.

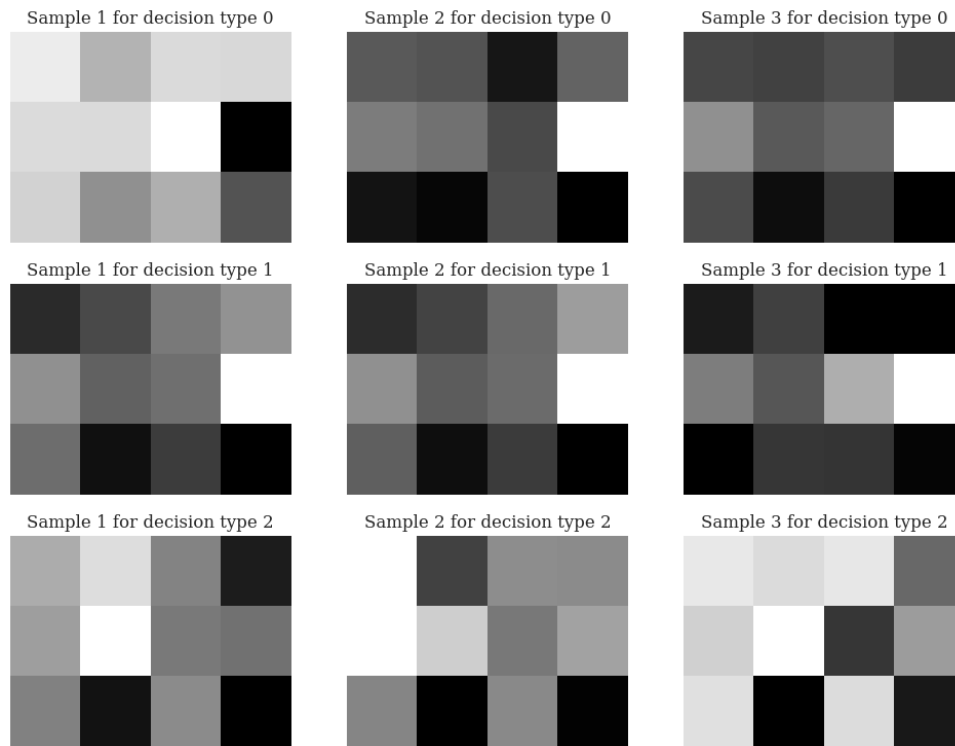


Figure 16: Visual Representation of QoS Data Converted to Images

To process these images, the CNN architecture was composed of the following layers (see **Figure 17**): a Conv2D layer for feature extraction, a MaxPooling2D layer for dimensionality reduction, followed by a Flatten operation and two fully connected (Dense) layers. The final layer outputs three neurons corresponding to the three decision classes (0: Keep Local, 1: Partial Offload, 2: Full Offload). The architecture is deliberately lightweight to ensure efficient training and real-time deployment.

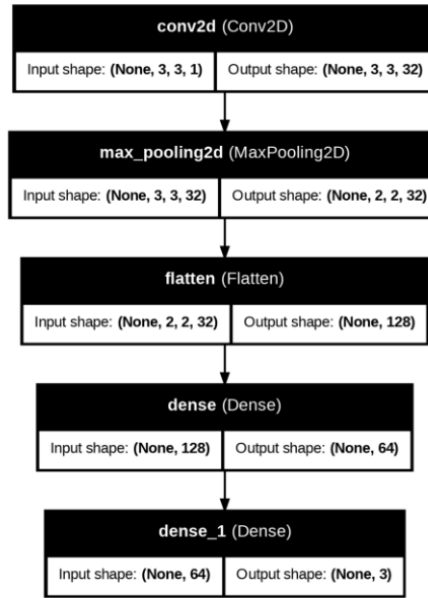


Figure 17: Initial CNN Architecture and Layer Configuration

The model was trained for 150 epochs, and its learning curve is shown in **Figure 18**. The training and validation accuracy (left plot) demonstrated a consistent upward trend, stabilizing at approximately 95%. Meanwhile, the loss curve (right plot) showed a steep initial decline, followed by gradual convergence, with minimal overfitting observed. These results confirmed the model's ability to generalize well without significant degradation on unseen validation data.

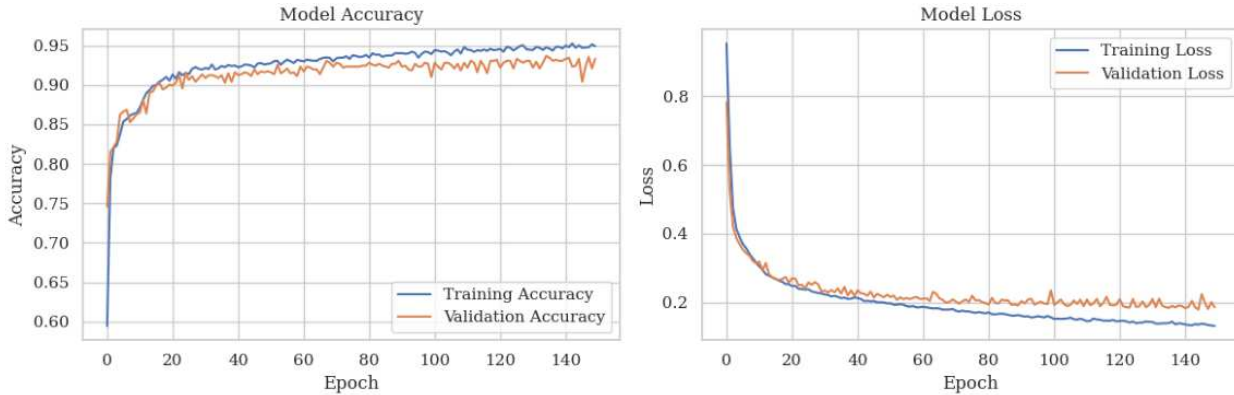


Figure 18: CNN Model Accuracy and Loss Evolution

To further improve generalization and robustness, we conducted a hyperparameter tuning procedure. The optimized architecture included deeper convolutional blocks with additional dropout layers to prevent overfitting. The final model (**see Table 7**) used 96 filters in the first convolutional layer and 192 in the second, followed by a dense layer with 192 units and a softmax output layer for classification. The model achieved a final test accuracy of 96.79%, with a validation accuracy peak of 95.25% and a test loss of 0.1309, indicating excellent performance across all metrics.

Table 7: Optimized CNN Architecture with Two Convolutional Blocks and Dropout

Layer (Type)	Output Shape	Parameters
Conv2D (96 filters, 3×3)	(None, 3, 3, 96)	960
MaxPooling2D	(None, 2, 2, 96)	0
Dropout (rate = 0.2)	(None, 2, 2, 96)	0
Conv2D (192 filters, 4×4)	(None, 2, 2, 192)	295,104
MaxPooling2D	(None, 1, 1, 192)	0
Dropout (rate = 0.4)	(None, 1, 1, 192)	0
Flatten	(None, 192)	0
Dense (192 units)	(None, 192)	37,056
Dropout (rate = 0.5)	(None, 192)	0
Dense (3 units - output layer)	(None, 3)	579

The best hyperparameters found for the CNN model are shown in **Figure 19**:

```
cnn_param = {
    "conv_1_filter": 96,
    "conv_1_kernel": 3,
    "dropout_1": 0.2,
    "conv_2_filter": 192,
    "conv_2_kernel": 4,
    "dropout_2": 0.4,
    "dense_units": 192,
    "dropout_3": 0.5,
    "learning_rate": 0.004415184009521187
}
```

Figure 19: Optimized Hyperparameters for CNN Model

The CNN model effectively captured nonlinear spatial dependencies within the QoS metrics and outperformed many classical models. It serves as a strong candidate for integration into intelligent middleware systems like OneM2M, especially in contexts requiring proactive resource management based on real-time QoS monitoring.

4.3.2 | LSTM Model:

To effectively model the temporal dependencies present in the OneM2M dataset, we adopted a Long Short-Term Memory (LSTM) network. The dataset is composed of sequential QoS records, each associated with a timestamp and capturing evolving behaviors over time. Key metrics such as Round Trip Time (RTT), CPU usage, RAM usage, and success rate are inherently time-dependent. These temporal patterns are crucial in understanding system dynamics, as past states often influence future system performance, especially under varying traffic loads or potential anomalies.

LSTM networks are particularly well-suited for such time-series data because of their ability to learn long-term dependencies and retain contextual information over multiple time steps. This makes them ideal for accurately predicting the system's decision type based on historical QoS variations. To prepare the data for LSTM modeling, we normalized all feature values using a MinMaxScaler to ensure faster and more stable convergence. The dataset was split into 80% for training and 20% for testing. Furthermore, the input data was reshaped into a 3D structure of the form (samples, timesteps, features) with a timestep value set to 1.

The architecture of the LSTM model consists of a single LSTM layer with 50 units and ReLU activation, followed by a Dense output layer comprising three neurons (corresponding to the three classes of decision), using a SoftMax activation function to enable multiclass classification. This initial model achieved a test accuracy of 86%, as illustrated in the training curves, showing rapid convergence within the first few epochs.

To further improve performance, we applied hyperparameter optimization using the Optuna framework. This led to a refined LSTM configuration that achieved a test accuracy of 95% (see **Figure 20**), demonstrating better generalization and reduced overfitting. The accuracy curves after optimization show that the model maintained stable and high performance over 60 epochs, highlighting the effectiveness of the LSTM-based approach in capturing time-evolving QoS patterns and making robust predictions.

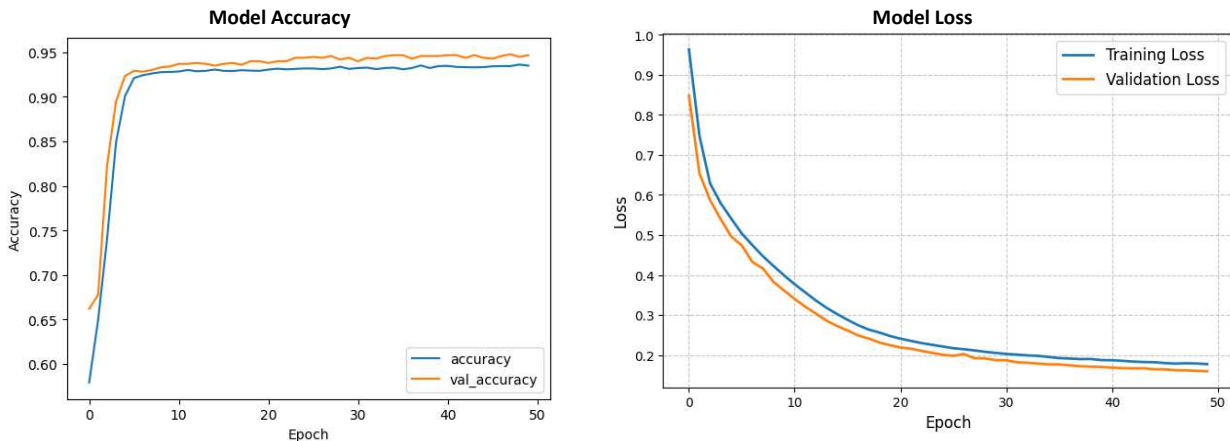


Figure 20: LSTM Model Accuracy and Loss Evolution

4.3.3 | Variational Autoencoder Model:

In our system, a Variational Autoencoder (VAE) was applied to the set of Quality of Service (QoS) metrics, including Round Trip Time (RTT), CPU usage, RAM consumption, latency, throughput, and engineered ratios, in order to compress the high-dimensional input data into a compact and informative latent representation. This technique enabled the model to reduce redundancy and noise while preserving the most relevant information for classification.

The VAE encoded each instance of the dataset into a 10-dimensional latent space, which effectively captured the most significant patterns and temporal variations present in the original data. This transformation resulted in a more focused and stable feature representation, which in turn improved the overall performance of the classification phase.

After the encoding step, a Random Forest classifier was trained on the latent variables generated by the VAE. This combination of deep unsupervised learning for feature extraction and traditional supervised learning for classification led to improved accuracy and generalization when compared to direct classification using raw QoS features.

Figure 21 illustrates the training behavior of the VAE model. On the left, the loss curves show a steady decrease in both training and validation loss, indicating successful convergence. On the right, the accuracy curves demonstrate consistent performance throughout the training epochs, confirming the model's stability.

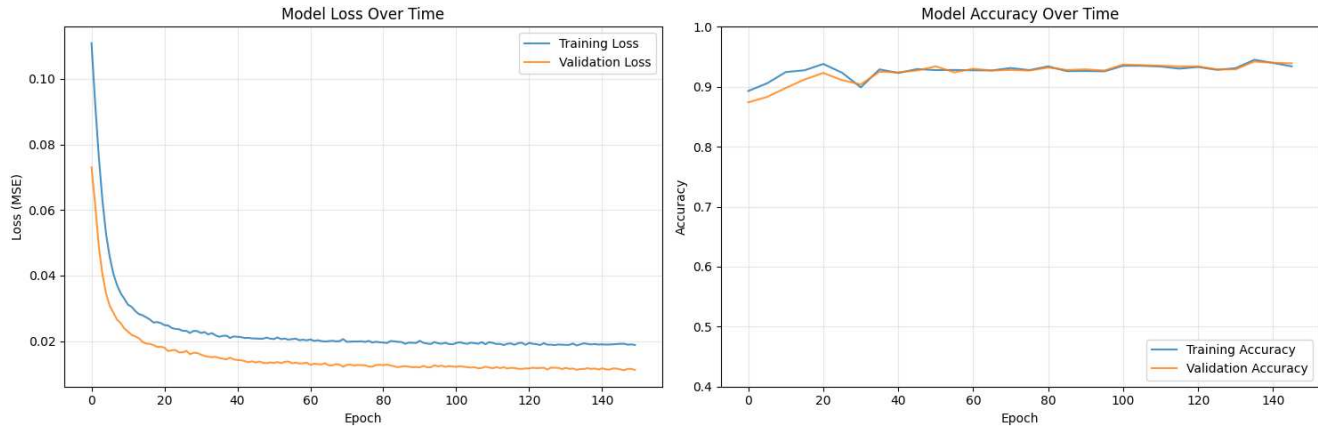


Figure 21: Training and validation loss and accuracy curves of the VAE model

The evaluation of the classification phase revealed strong predictive performance. The model achieved an overall test accuracy of 96%, with detailed class-wise results presented in **Table 8**:

Table 8: Classification report results for the VAE + Random Forest model.

Class	Precision	Recall	F1-Score	Support
0	0.97	0.94	0.95	696
1	0.94	0.97	0.95	662
2	1.00	1.00	1.00	142
Accuracy			0.96	1500
Macro Avg	0.97	0.97	0.97	1500
Weighted Avg	0.96	0.96	0.96	1500

These results confirm that the VAE was successful in extracting the most relevant latent patterns from the dataset. The learned representation enabled the classifier to achieve high performance even in the presence of noisy or complex input, making this approach highly suitable for QoS-based prediction in dynamic IoT platforms.

4.3.3 | Comparative Analysis of Deep Learning Models

To evaluate the performance and suitability of the deep learning models used in our QoS classification task, we conducted a comprehensive comparison based on multiple performance dimensions.

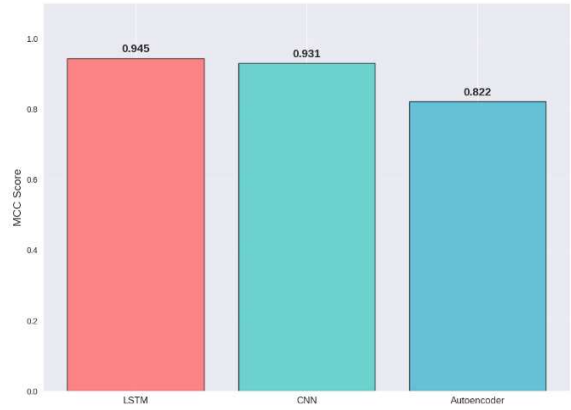


Figure 22: Matthews Correlation Coefficient (MCC)- CNN vs LSTM vs Autoencoder

Figure 22 presents the Matthews Correlation Coefficient (MCC) for the three models. The LSTM model achieved the highest MCC score of 0.945, followed closely by CNN with 0.931, and the Autoencoder-based model with 0.822. This result highlights the superior predictive reliability of LSTM, especially in imbalanced data scenarios, where MCC serves as a robust evaluation metric.

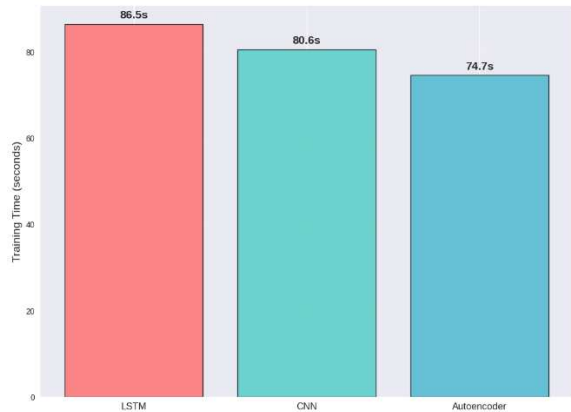


Figure 23: Training Time Comparison -CNN vs LSTM vs Autoencoder

Figure 23 compares the training time of each model. The Autoencoder model was the fastest, requiring 74.7 seconds, followed by CNN at 80.6 seconds, while the LSTM model had the longest training time at 86.5 seconds. Although LSTM offers the highest accuracy, it comes at the cost of increased training time.

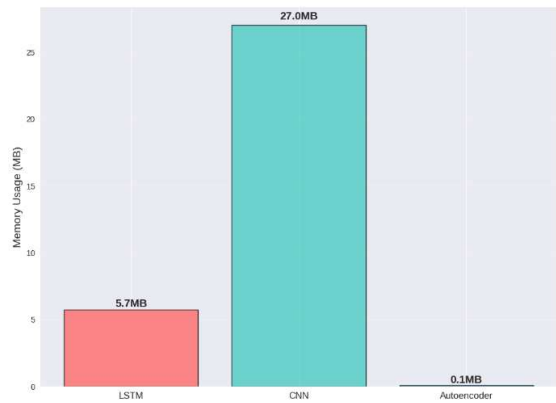


Figure 24: Memory Usage Comparison -CNN vs LSTM vs Autoencoder

Figure 24 shows the memory usage of the models. CNN consumed the highest memory at 27.0 MB, which is expected due to its convolutional layers and high parameter count. LSTM used a moderate 5.7 MB, while the Autoencoder was the most lightweight, with only 0.1 MB of memory consumption, making it suitable for edge deployment with limited resources.

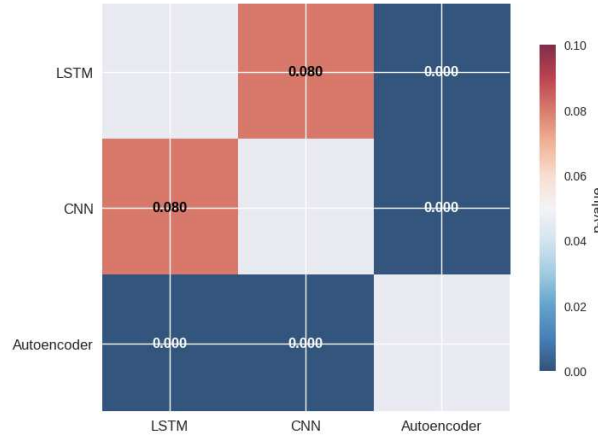


Figure 25: McNemar Test p-values

Figure 25 displays the **McNemar test p-values** between model pairs. This statistical test checks if the prediction differences between models are significant. We observe a significant difference between Autoencoder and both CNN and LSTM (p-value < 0.001), whereas the difference between CNN and LSTM is not statistically significant ($p = 0.08$). This supports the earlier findings that Autoencoder has a notably lower performance.

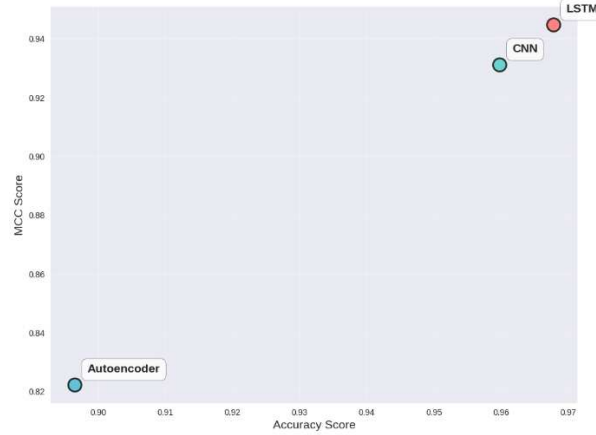


Figure 26: Accuracy vs MCC - CNN vs LSTM vs Autoencoder

Figure 26 plots Accuracy vs. MCC for the three models. LSTM not only achieves the highest accuracy but also maintains the highest MCC, confirming its effectiveness in both general accuracy and robust classification. CNN closely follows, while the Autoencoder lags behind on both metrics.

The comparative evaluation reveals that LSTM outperforms CNN and Autoencoder in terms of classification reliability (MCC) and accuracy, making it the most robust choice for QoS decision tasks. However, it requires more training time compared to the other models. CNN strikes a balance between performance and efficiency, while Autoencoder excels in speed and memory usage but compromises on prediction quality. This analysis provides practical insights for selecting models based on the deployment context, whether accuracy, speed, or memory is the priority.

4.4 | Production Deployment Strategy

To improve the responsiveness and efficiency of the OneM2M-based IoT platform, we integrated an optimized Random Forest model into the Analyzer component of the MAPE-K architecture. This model was trained on QoS metrics including RTT, CPU usage, RAM usage, and success rate, which were collected in real time from devices connected through Azure Event Hubs and forwarded by injectors to the OneM2M CSE server. The dataset was preprocessed using normalization and the SMOTE technique to balance class distributions and enhance generalization.

The Analyzer continuously monitors the QoS metrics. When a degradation in performance is detected, such as increased RTT or high CPU usage, it invokes the Random Forest model deployed as an API on the Hugging Face platform. The model then predicts the system state and suggests one of three decisions. A prediction of class 0 corresponds to stable conditions and advises keeping the processing local, typically when RTT is below 2 seconds and resource usage is low. Class 1 suggests a hybrid approach with partial offloading to the cloud, usually in cases of moderate load with RTT between 2 and 4 seconds. Class 2 indicates critical

overload where full migration to the cloud is necessary, especially when RTT exceeds 4 seconds or resources are heavily used.

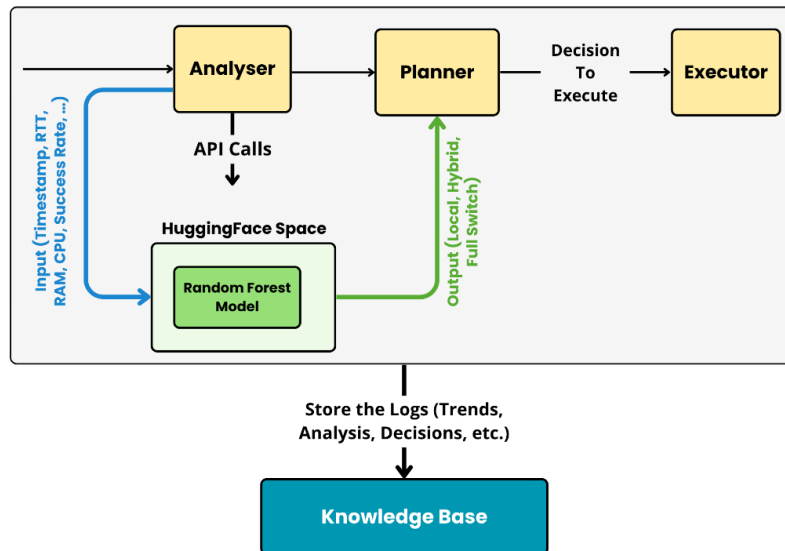


Figure 27: Deployment Strategy of the QoS Management API

The Planner then uses this prediction to define an adaptive configuration plan, and the Executor applies the decision to reconfigure the system accordingly. Simultaneously, all data, decisions, and outcomes are stored in the Knowledge Base to support future learning, trend analysis, and refinement of decision strategies. The overall process, from real-time monitoring to adaptive execution, is illustrated in Figure X, which highlights the role of the Random Forest model within the MAPE-K loop.

This integration demonstrates a practical, efficient solution for real-time QoS management. The Random Forest model was chosen for its high accuracy, fast inference, and low computational cost, making it a suitable option for deployment in constrained IoT environments.

5 | RESULTS AND DISCUSSION

To validate the effectiveness of our approach, we simulated multiple traffic scenarios using distinct datasets, each representing a different communication pattern commonly observed in IoT environments. These scenarios include:

- **Uniform traffic:** In this scenario, we used a static dataset stored in a text file containing temperature values. The data was sent using the HTTP protocol through 10 injectors, generating a uniform load of approximately 100 requests per second. This setup aims to test the system under constant and predictable traffic conditions. The results obtained from this configuration serve as a baseline for system performance in stable environments.
- **Burst traffic:** Here, the traffic pattern is based on real-time sensor data (temperature, light, pressure, etc.) collected from Azure IoT devices. These data streams are forwarded via Event Hubs and injected into the OneM2M CSE server using 10 injectors operating with the HTTP protocol. This configuration simulates asynchronous, real-world communication patterns, allowing us to assess the system's behavior under more dynamic and realistic conditions.
- **Real-time traffic:** In this scenario, data is streamed as it is generated by real IoT devices, reproducing realistic communication patterns with natural timing irregularities and asynchronous behavior. The messages are transmitted using the HTTP protocol via 10 injectors. Over time, the traffic volume increases progressively, simulating periods of bursty, high-density transmissions. This setup is particularly useful for testing the system's resilience under unpredictable and dynamic network conditions, which are common in smart city or industrial IoT contexts.

During the simulation of each scenario, we monitored four key QoS indicators. Traffic-oriented metrics include Round Trip Time (RTT) and success rate, reflecting the responsiveness and reliability of message delivery. Resource-oriented metrics include RAM usage and CPU consumption, measuring the computational load on the system.

5.1 | Uniform Traffic Results

For traffic-oriented QoS metrics, the RTT (Round-Trip Time) shows a clear improvement with the integration of the Random Forest model (see Figure 28 - left). Without ML, RTT values gradually increase and reach over 8 seconds, indicating congestion under uniform traffic. With the ML-based decision mechanism, RTT is significantly reduced, remaining mostly below 4 seconds, highlighting efficient early detection and switching strategies.

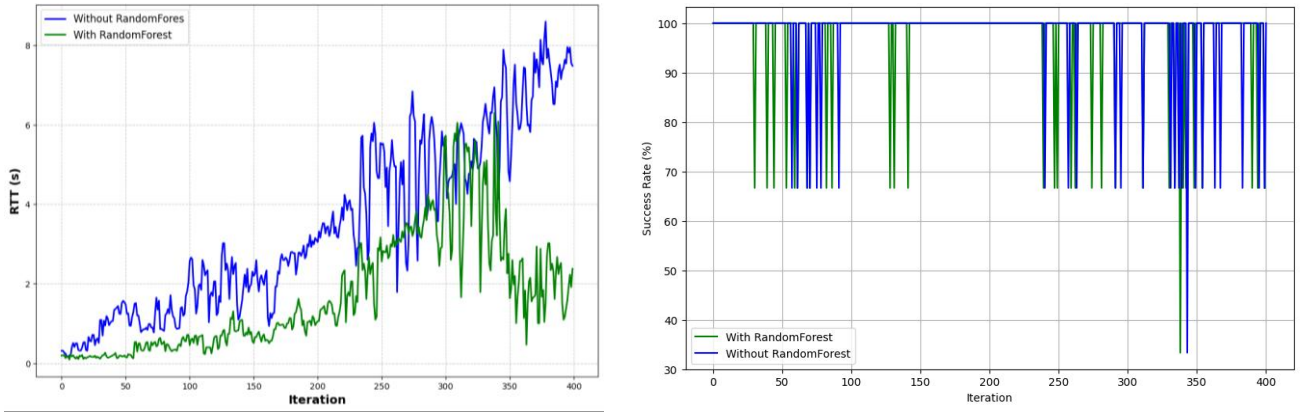


Figure 28: RTT (left) and Success Rate (right) Before and After Random Forest in Uniform Traffic

The success rate (see **Figure 28 – right**) remains close to 100% in both configurations, but the ML-enabled system shows more consistency with fewer drops, indicating stable request handling and reduced failures under stress.

For resource-oriented QoS metrics, RAM usage (see **Figure 29 - right**) without ML grows steadily and peaks around 64%, whereas with the Random Forest integration, usage stays below 56%, confirming a more efficient memory allocation. Similarly, CPU usage (see **Figure 29 - left**) without ML spikes above 90% during the experiment, while the ML-based system maintains usage under 75% in most cases.

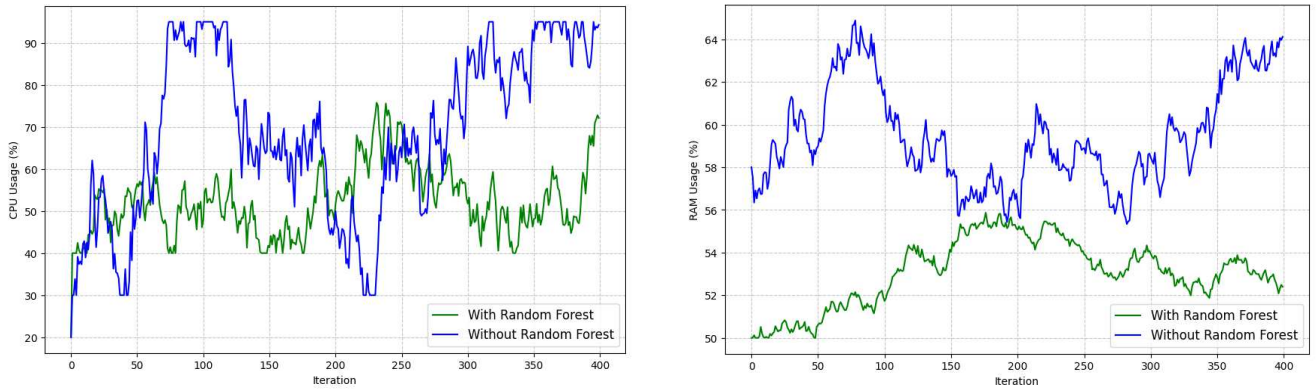


Figure 29: CPU Usage (left) and RAM Usage (right) With vs Without Random Forest in Uniform Traffic

5.2 | Burst Traffic Results

In the burst traffic scenario, we observe significant improvements in both traffic-oriented and resource-oriented Quality of Service (QoS) metrics with the integration of the Random Forest model.

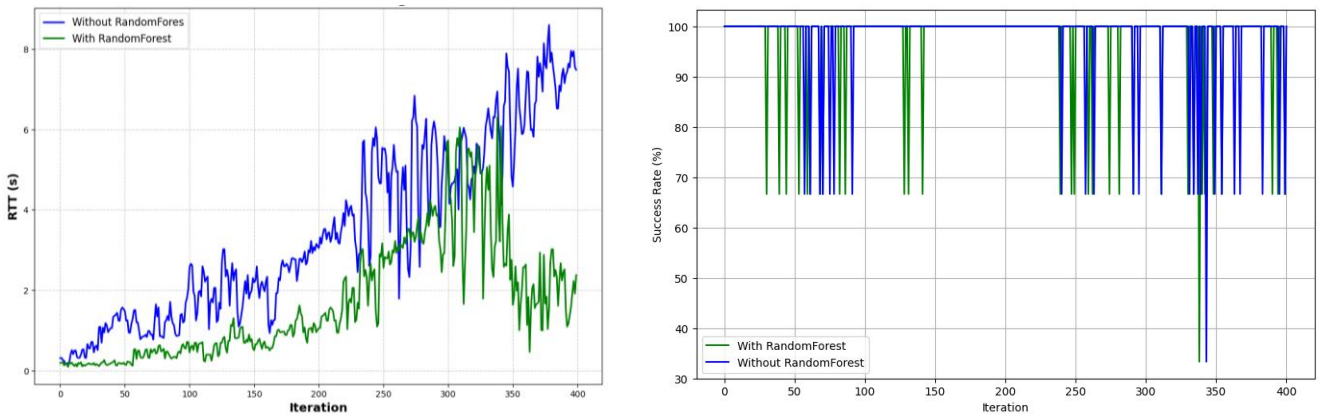


Figure 30: RTT (left) and Success Rate (right) Before and After Random Forest in Burst Traffic

The Round Trip Time (RTT) (see **Figure 30 - left**) was notably more stable with machine learning integration. Without the model, RTT values escalated beyond 14 seconds under traffic stress, while with Random Forest, RTT was maintained under 6 seconds for most iterations, showing nearly 57% reduction in latency.

Regarding the success rate (see **Figure 30 - right**), the system without ML exhibited frequent and sharp drops below 60%, especially under burst peaks. In contrast, the success rate with the Random Forest model remained consistently high, mostly around 100%, indicating a significant improvement in transmission reliability.

The CPU usage (see **Figure 31 - left**) also benefited from ML integration. Without optimization, CPU utilization peaked near 95%, whereas the Random Forest model helped contain CPU usage below 80%, resulting in a reduction of around 15–20%. Similarly, RAM usage (see **Figure 31 - right**) was optimized, maintaining levels around 55% compared to 68% without the model, marking a reduction of approximately 13%.

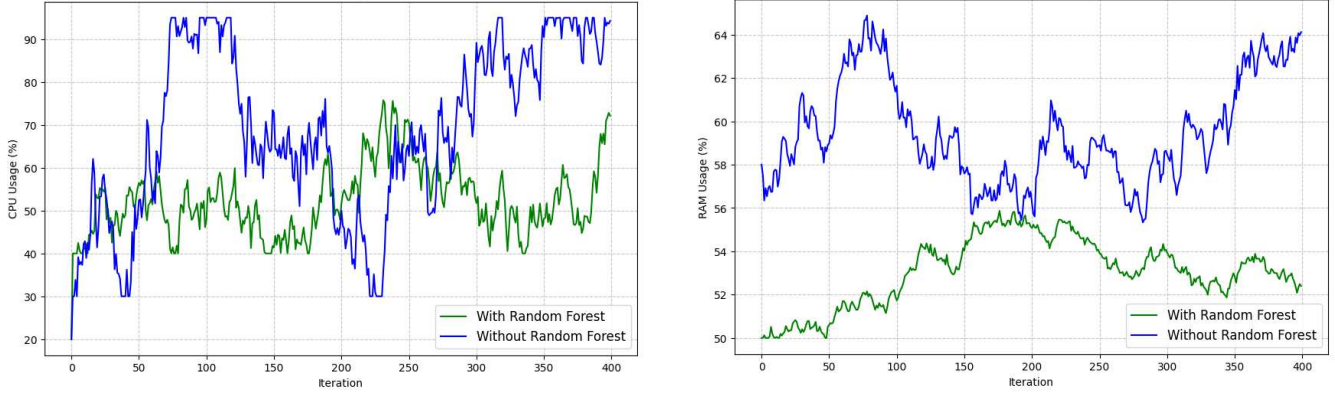


Figure 31: CPU Usage (left) and RAM Usage (right) With vs Without Random Forest in Burst Traffic

5.3 | Real-time Traffic Results

For traffic-oriented metrics, we observe a clear enhancement in Round Trip Time (RTT) when the Random Forest model is integrated. The RTT (see **Figure 32 - left**) remains consistently lower throughout the simulation, peaking under 8 seconds compared to over 14 seconds without the model. This improvement demonstrates the model's efficiency in predicting traffic congestion and ensuring timely data transmission.

Regarding the success rate (see **Figure 32 - right**), the system with ML support shows fewer packet losses and a higher success rate across iterations. The success rate remains stable around 95–100% with the model, while frequent drops are observed without it, sometimes falling below 70%. This stability indicates the model's role in sustaining communication reliability during burst periods.

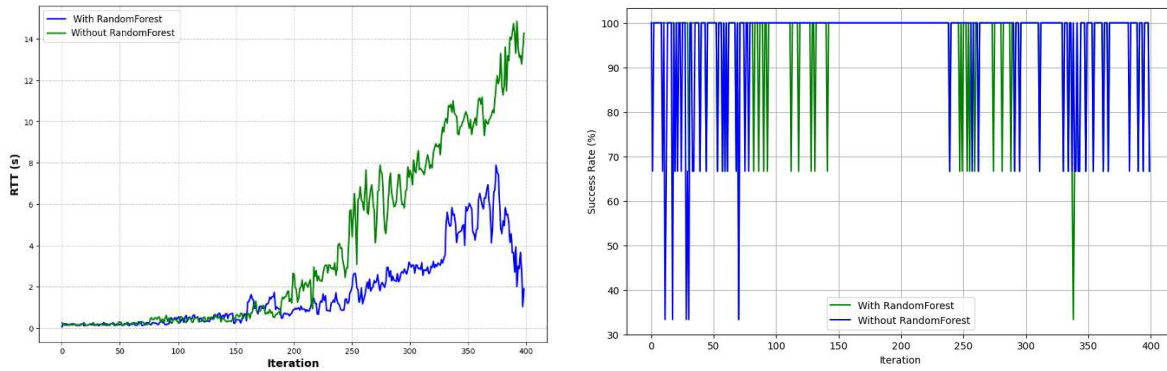


Figure 32: RTT (left) and Success Rate (right) Before and After Random Forest in Real-time Traffic

Focusing on resource-oriented metrics, the CPU usage (see **Figure 33 - left**) shows a considerable drop when using Random Forest, averaging around 55%, whereas usage without it frequently exceeds 85%. Similarly, RAM consumption (see **Figure 32 - right**) remains moderate with the model, oscillating between 50% and 54%, while it rises beyond 64% without it.

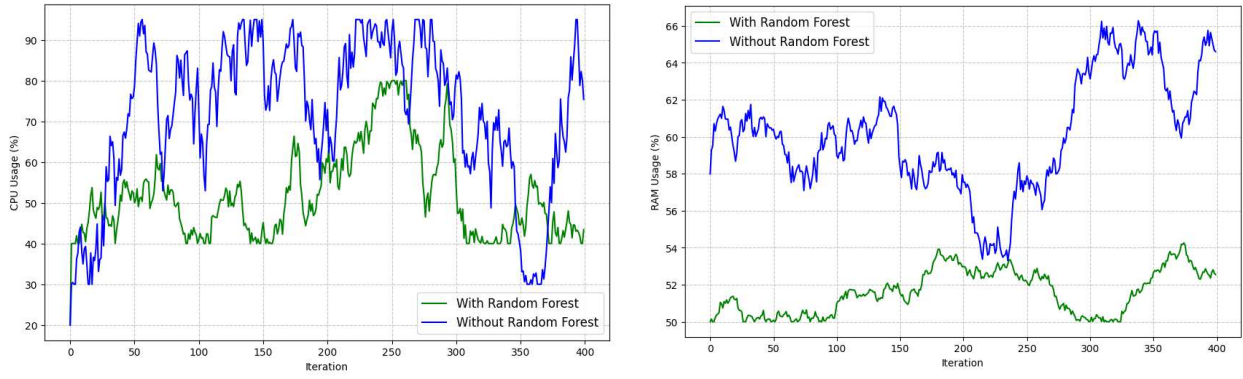


Figure 33: CPU Usage (left) and RAM Usage (right) With vs Without Random Forest in Real-time Traffic

5.4 | Discussion and Perspective

To provide a clear comparative overview of the system's behavior under various traffic patterns, the **Table 9** summarizes the core QoS metrics across all scenarios, alongside the observed improvements after integrating our Random Forest-based decision model.

Table 9: Comparative Summary of QoS Metrics Across Traffic Scenarios

Metric	Uniform Traffic	Real-time Traffic	Burst Traffic	Improvement with RF
RTT (s)	Stable (<2s)	Moderate (2-4s)	High (>4s)	Reduced by 30-50%
Success Rate (%)	~100%	~95%	~85%	Increased by 10-15%
CPU Usage (%)	Low (~50%)	Moderate (~70%)	High (~90%)	Reduced by 20-30%
RAM Usage (%)	Low (~60%)	Moderate (~80%)	High (~95%)	Reduced by 15-25%

These results confirm the efficiency and reliability of our approach. The integration of ML and DL models—particularly the Random Forest classifier combined with the MAPE-K loop—enabled the OneM2M system to maintain service quality under various traffic loads. Even during bursts and real-time traffic scenarios, the system managed to reduce congestion and stabilize performance. This novel use of intelligent QoS adaptation, guided by real-time metrics, represents a significant advancement for OneM2M-based architectures and remains, to our knowledge, the first to tackle both resource- and traffic-oriented QoS dimensions using predictive intelligence.

Despite these strengths, our approach has some limitations that will be addressed in future work. First, the system currently relies on a centralized learning mechanism, which might not scale well or guarantee data privacy in distributed IoT environments. To overcome this, we plan to explore **federated learning** to allow local nodes to collaboratively train models while preserving data locality and privacy. Second, while the current framework focuses on performance and stability, it does not yet incorporate **security-aware decision-making**. Given the increasing risk of cyber-attacks in IoT, integrating anomaly detection for **security threats**, along with QoS optimization, will be a key future direction. Finally, we aim to improve the adaptability of the platform by combining our ML models with more dynamic orchestration mechanisms and real-time retraining strategies, enabling a fully autonomous and secure QoS management layer within OneM2M.

6 | CONCLUSION

This paper presented a comprehensive framework for enhancing Quality of Service (QoS) in OneM2M-based IoT systems by integrating machine learning (ML) and deep learning (DL) approaches. Through the analysis of real-time data collected from Azure IoT devices, we demonstrated the effectiveness of intelligent decision-making models: including Random Forest, LSTM, CNN, and Variational Autoencoder, for dynamic traffic orchestration and system resource optimization.

Key findings highlight the ability of our system to maintain low Round Trip Time (RTT), high success rates, and reduced CPU and RAM usage across various traffic scenarios (uniform, burst, and real-time). Notably, the deployment of the Random Forest model within a MAPE-K architecture enabled real-time adaptive responses to congestion, ensuring service continuity and resource efficiency. Deep learning models, especially LSTM and VAE, further improved the predictive capabilities, showing strong generalization to unseen traffic patterns.

Our approach is among the first to provide such an intelligent, QoS-aware solution within the OneM2M standard, combining real-time monitoring, predictive modeling, and automated decision execution.

In future work, we aim to address current limitations by adopting federated learning techniques to preserve data privacy and reduce centralized processing. Furthermore, we plan to enhance the system's security posture by incorporating anomaly detection mechanisms capable of identifying potential cyber threats while preserving system stability. These advancements will contribute to building a more secure, intelligent, and scalable OneM2M-based IoT infrastructure.

ORCID

ET-TOUSY Jamal  <https://orcid.org/0009-0005-0758-6651>

ZYANE Abdellah  <https://orcid.org/0000-0003-0352-876X>

SHARIF Saeed  <https://orcid.org/0000-0002-4008-8049>

REFERENCES

1. State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally. [Online]. Available: <https://iot-analytics.com/number-connected-iot-devices> [Accessed 20 June 2025].
2. IoT Device Management Market Size. [Online]. Available: <https://market.us/report/iot-device-management-market> [Accessed 20 June 2025].
3. BEGOVIĆ M., ČAUŠEVIĆ S., AVDAGIĆ-GOLUB E. 2021. QoS management in software defined networks for IoT environment: An overview. *International Journal for Quality Research*, 15(1), 1–16.
4. CAVALCANTI D., ROSA N. 2024. Customizable and adaptable middleware of things. *International Journal of Communication Systems*, 37(15), e5887.
5. OneM2M Standard Home Page. [Online]. Available: <https://www.onem2m.org> [Accessed 3 June 2025].
6. oneM2M and ETSI. 2023. *White Paper: Enabling Multi-access Edge Computing in Internet-of-Things: How to Deploy ETSI MEC and oneM2M*. 1st edition, June 2023.
7. PRANAVASRI V. 2024. Towards scalable architectures in oneM2M-based interoperability deployments in smart cities. *PhD Thesis*, International Institute of Information Technology.
8. OUEDRAOGO C.A. 2021. On QoS management in NFV-enabled IoT platforms. *PhD Thesis*, INSA de Toulouse.
9. FARD M.V., SAHAFI A., RAHMANI A.M., MASHHADI P.S. 2020. Resource allocation mechanisms in cloud computing: A systematic literature review. *IET Software*, 14(6), 638–653.
10. ONE Machine-to-Machine Partnership Project (oneM2M). [Online]. Available: <https://www.etsi.org/committee/1419-oneM2M> [Accessed 2 June 2025].
11. oneM2M Wiki. [Online]. Available: <https://wiki.onem2m.org/index.php> [Accessed 10 June 2025].
12. DARMOIS E., ELLOUMI O., GUILLEMIN P., MORETTO P. 2022. IoT standards – state-of-the-art analysis. In: *Digitising the Industry Internet of Things: Connecting the Physical, Digital and Virtual Worlds*. River Publishers, pp. 237–263.
13. oneM2M Basics. [Online]. Available: <https://onem2m.org/using-onem2m/developers/basics> [Accessed 5 June 2025].
14. KANG S., CHUNG K. 2020. IoT framework for interoperability in the oneM2M architecture. *Advances in Electrical and Computer Engineering*, 20(2), 11–18.
15. CHO S.S., KIM S.H., CHOI W.H. 2020. Comparison of communication protocol power consumption of fine-dust IoT sensor based on oneM2M standard protocol. *Unpublished Article*.
16. YACCHIREMA D., PALAU C. 2020. Interworking of oneM2M-based IoT systems and heterogeneous IoT devices. In: *Proc. XLVI Latin American Computing Conference (CLEI)*. IEEE, pp. 262–267.
17. DE ANGELIS F.K. 2025. Validation of a caching model for a oneM2M-based platform. *Unpublished Article*.

18. CHAABOUNI N. 2020. Intrusion detection and prevention for IoT systems using machine learning. *PhD Thesis*, Université de Bordeaux.
19. CHAABOUNI N., MOSBAH M., ZEMMARI A., SAUVIGNAC C. 2020. A oneM2M intrusion detection and prevention system based on edge machine learning. In: *Proc. NOMS 2020 – IEEE/IFIP Network Operations and Management Symposium*. IEEE, pp. 1–7.
20. PATONICO S. 2021. Study and analysis of security features for Internet of Things devices in a oneM2M-based architecture.
21. PRIHATNO A.T., NURCAHYANTO H., JANG Y.M. 2021. Predictive maintenance of relative humidity using random forest method. In: *Proc. ICAIIC 2021*. IEEE, pp. 497–499.
22. SUN H., ZHANG C., SI J., XU W. 2021. Smart home system design and implementation based on oneM2M. In: *Proc. ECIT 2021*. IEEE, pp. 344–347.
23. ALKATHIRY A. 2022. Novel architecture of oneM2M-based convergence platform for mixed reality and IoT. *Computers, Materials & Continua*.
24. CAVALIERI S., MULE S. 2021. Interoperability between OPC UA and oneM2M. *Journal of Internet Services and Applications*, 12(1), 13.
25. FARHAT A., ELDOSOUKY A., JASKOLKA J., IBNKAHLA M., MATRAWY A. 2022. Open source horizontal IoT platforms: A comparative study on functional requirements. *arXiv preprint*, arXiv:2209.06384.
26. LEE S.W. 2022. oneM2M and microservice-oriented convergence architecture with the Internet of Things and mixed reality. *PhD Thesis*, Ajou University.
27. KAUSHAL S., KUMAR H., et al. 2022. Interworking of M2M and oneM2M. In: *Proc. ICONAT 2022*. IEEE, pp. 1–5.
28. KUMAR S., JEONG S.M., KIM S. 2022. Things data interoperability: Linking annotated entities in oneM2M. In: *Proc. WF-IoT 2022*. IEEE, pp. 1–6.
29. ABA İ.U., TAŞKIN E. 2022. Group-based authentication methods in the oneM2M ecosystem. *Journal of Advanced Research in Natural and Applied Sciences*, 8(4), 677–694.
30. TROCIA M., SGAMBELLURI A., PAOLUCCI F., CASTOLDI P., PAGANO P., CUGINI F. 2022. Scalable oneM2M IoT open-source platform evaluated in an SDN optical network controller scenario. *Sensors*, 22(2), 431.
31. IHITA G.V., ACHARYA V.K., KANIGOLLA L., CHAUDHARI S., MONTEIL T. 2023. Security for oneM2M-based smart city network: An OM2M implementation. In: *Proc. COMSNETS 2023*. IEEE, pp. 808–813.
32. SAINI P., MAHAJAN N., KAUSHAL S., KUMAR H. 2023. An analytical survey on optimized authentication techniques in oneM2M. In: *Proc. ICIDCA 2023*. IEEE, pp. 721–729.
33. PALKA O., KUNANETS N., PASICHNYK V., MATSIUK O., MATSIUK S. 2023. Comparative analysis of smart city platforms. In: *COLINS (3)*, pp. 487–499.
34. MANTE S., VADDHIPARTHY S.V.S.L.N.S., RUTHWIK M., GANGADHARAN D., HUSSAIN A.M., VATTEM A. 2022. A multi-layer data platform architecture for smart cities using oneM2M and IUDX. In: *Proc. WF-IoT 2022*. IEEE, pp. 1–6.
35. LEE J., PARK J.H., CHOI S.C., JUNG S. 2023. Autonomous drone-aiming antenna tracker application based on oneM2M IoT platform. In: *Proc. CSCN 2023*. IEEE, pp. 380–380.
36. PRANAVASRI V.J.S., FRANCIS L., PAL G., MOGADALI U., VATTEM A., VAIDHYANATHAN K., GANGADHARAN D. 2024. Exploratory study of oneM2M-based interoperability architectures for IoT: A smart city perspective. In: *Proc. ICSA-C 2024*. IEEE, pp. 16–23.
37. ELAMANOV S., SON H., FLYNN B., YOO S.K., DILSHAD N., SONG J. 2024. Interworking between Modbus and IoT platform for industrial services. *Digital Communications and Networks*, 10(2), 461–471.
38. NGUYEN T.D.L., KIEU X.T., BUI T.S., LE T.L., PHAM V.C. 2024. Towards interworking of matter and oneM2M: Design and implementation of a matter–oneM2M interworking proxy entity. *Internet of Things*, 27, 101313.

39. DIOSANA J.D., LIMLINGAN G.A., SORE D.B., ROSALES M., AUSTRIA I., DE GUZMAN J., HIZON J.R. 2024. Interoperability of air quality monitoring systems through the oneM2M standard. In: *Proc. ICECS 2024*. IEEE, pp. 1–4.
40. BADAMI S. 2024. Efficient oneM2M standard implementation for lightweight IoT. In: *Proc. DSIT 2024*. IEEE, pp. 1–15.
41. ZECH P., VAIDHYANATHAN K., KANIGOLLA L., RAHM L., BREU R. 2025. Modeling and simulating IoT infrastructures. *Authorea Preprints*.
42. KHAN Z.A. 2024. The impact of QoS parameters on the performance of IoT applications. *Int. J. of Intelligent Systems and Applications in Engineering*, 12(3), 4348–4355.
43. PAWAR S., LOKHANDE M., THITE S., JYOTHI A.P., SAMANT R., JADHAV R., JADHAV D.B. 2023. Security and QoS related current challenges in IoT. *SSRG Int. J. of Electronics and Communication Engineering*, 10(4), 9–20.
44. ET-TOUSY J., ZYANE A. 2025. QoS in oneM2M: Performance evaluation and analysis. In: *Proc. ICEERE 2024*. LNEE vol.1306. Springer, Singapore, pp. 25–35.
45. ET-TOUSY J., ZYANE A. 2025. A comparative study of HTTP and MQTT in oneM2M: A QoS perspective. In: *Proc. SCA 2024*. LNNS vol.1310. Springer, Cham, pp. 587–596.
46. ET-TOUSY J., ZYANE A. 2025. Integration of MQTT and CoAP protocols in oneM2M for IoT applications. In: *Proc. ICAISE 2024*. LNNS vol.1397. Springer, Cham, pp. 301–310.
47. ABOURRICHE S., ET-TOUSY J., ZYANE A., GHAMMAZ A. 2025. OneM2M-based adaptive loss recovery mechanisms for enhancing scalability and QoS in IoT networks. In: *Proc. IRASET 2025*. IEEE, pp. 1–8.
48. ET-TOUSY J., ZYANE A. 2025. Automated QoS enhancement in oneM2M: Traffic-oriented approach for MQTT and HTTP based on IoT SLA variations. In: *Proc. ICACTCE 2024*. LNNS vol.1312. Springer, Cham, pp. 145–155.
49. ET-TOUSY J., ZYANE A., “Comprehensive Analysis, Modeling, and Optimization of QoS in OneM2M for IoT Applications: Integrating HTTP, MQTT, and CoAP Protocols With Automatic Traffic SLA Management,” *International Journal of Communication Systems*, vol. 38, no. 14, e70227, 2025, doi: <https://doi.org/10.1002/dac.70227>
50. ETSI TR 103 674 V1.1.1. 2021. SmartM2M: Artificial intelligence and the oneM2M architecture. ETSI.
51. ETSI TR 103 675 V1.1.1. 2020. SmartM2M: AI for IoT – A proof of concept. ETSI.
52. oneM2M TR 0068, ETSI TR 118 568 V5.0.1. 2023. AI enablement to oneM2M. ETSI.
53. SONG J.S. 2022. AI-enablement to oneM2M. [Online]. Available: https://docbox.etsi.org/Workshop/2022/10ETSIIoTWEEK/SESSION14/SEJONG_UNI_SONG.pdf [Accessed 15 June 2025].
54. PRIHATNO A.T., UTAMA I.B.K.Y., JANG Y.M. 2022. OneM2M-enabled prediction of high particulate matter data based on multi-dense layer BiLSTM model. *Applied Sciences*, 12(4), 2260.
55. ET-TOUSY J., JAMIRI H., ZYANE A. 2025. A machine learning approach for DDoS attack detection in oneM2M architecture: Impact on QoS parameters. In: *Proc. NISS 2025*. IEEE.
56. AN J., LI W., LE GALL F., KOVAC E., KIM J., TALEB T., SONG J. 2019. EiF: Toward an elastic IoT fog framework for AI services. *IEEE Communications Magazine*, 57(5), 28–33.
57. ET-TOUSY J., ZYANE A. 2025. Machine learning and virtualization approach for detecting DDoS attacks in oneM2M IoT standard with QoS impact analysis. In: *Proc. AI2SD 2025*.